Introduction to Scientific Programming

Lecture 1:

Introduction & Modern Development Ecosystem

Renato Duarte

@Renatocfduarte
comp-neuro.org
@rcfduarte
renato.duarte@cnc.uc.pt

CENTER FOR NEUROSCIENCE AND CELL BIOLOGY
UNIVERSITY OF COIMBRA
PORTUGAL

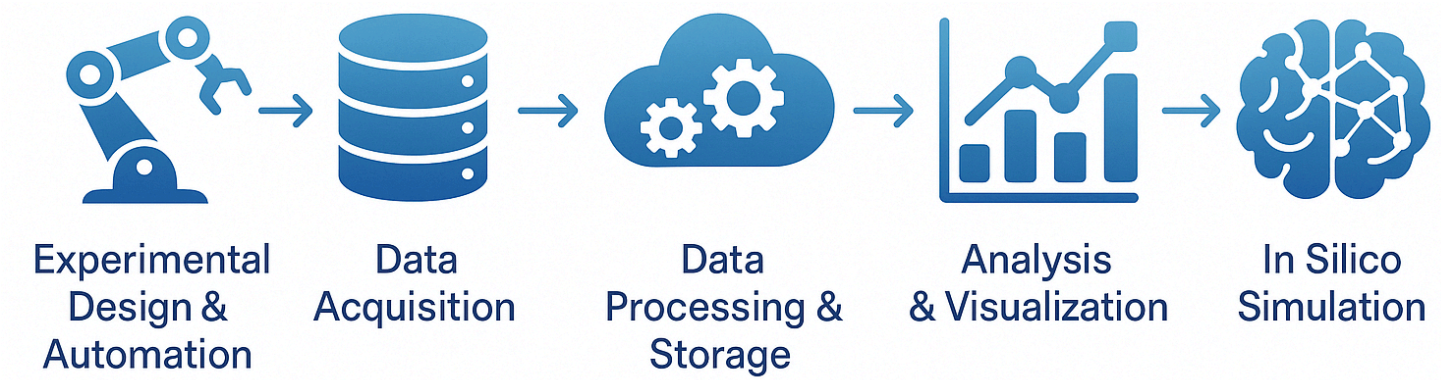
1 2 9 0 UNIVERSIDADE D COIMBRA

Introduction

Why scientific computing?

Scientific process is increasingly **computationally intensive** (all domains):

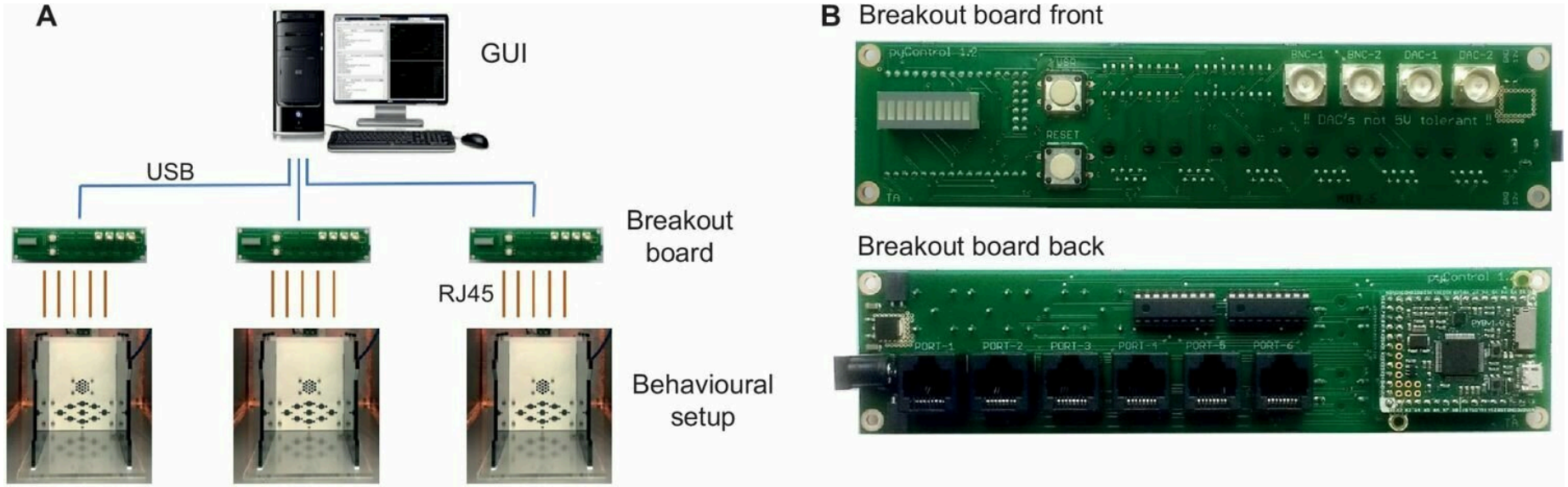
- ***Experimental design***, protocol implementation and ***automation***
- Experiment execution, ***data acquisition***
- Data ***storage***, curation and ***processing***
- Data ***analysis***, mining and visualization
- Numerical ***simulation*** and *in silico* hypothesis testing



Introduction

Why scientific computing?

- ***Experimental design***, protocol implementation and ***automation***
  - *Laboratory robotics, automation and high-throughput screening*
  - *Experimental planning, robotic sample preparation, protocol optimization*

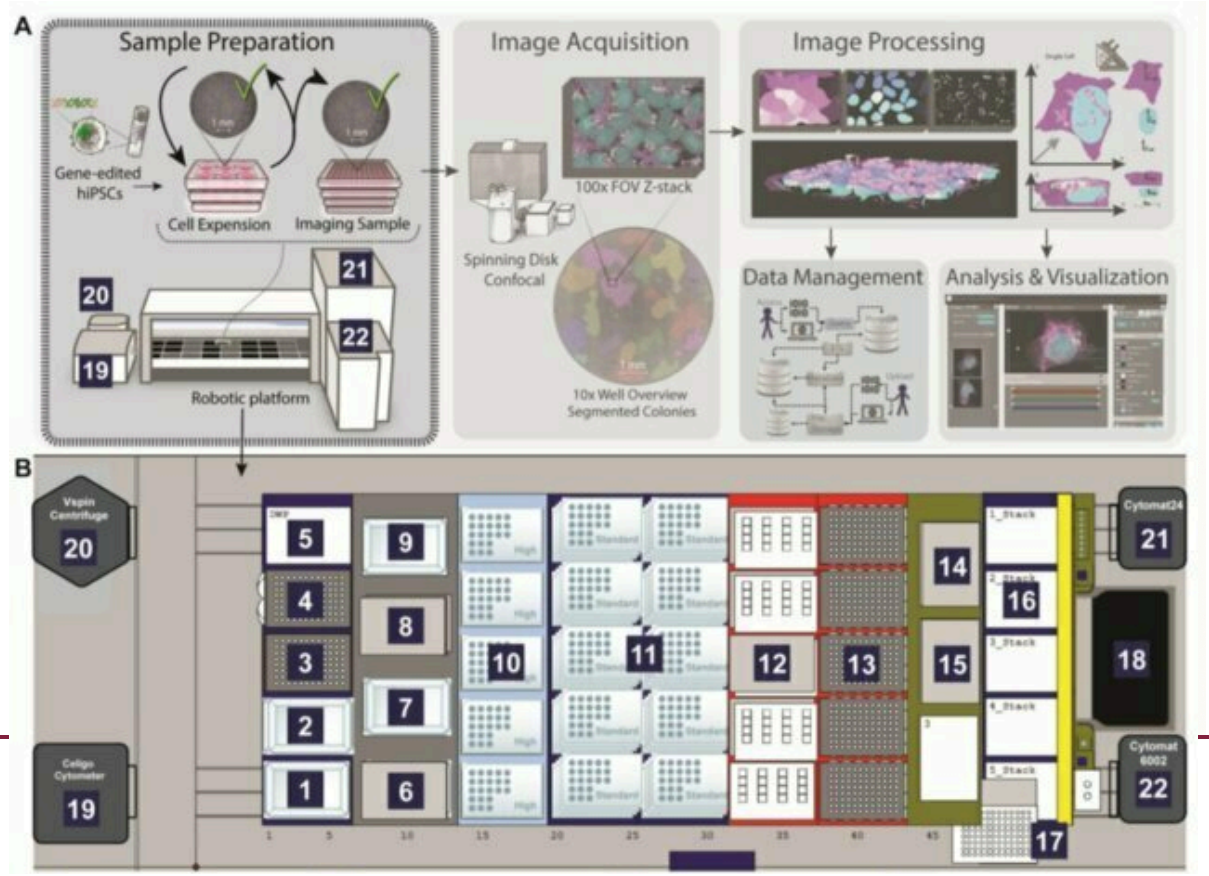


Thomas Akam, et al. (2022) Open-source, Python-based, hardware and software for controlling behavioural neuroscience experiments. eLife 11:e67846.

Introduction

Why scientific computing?

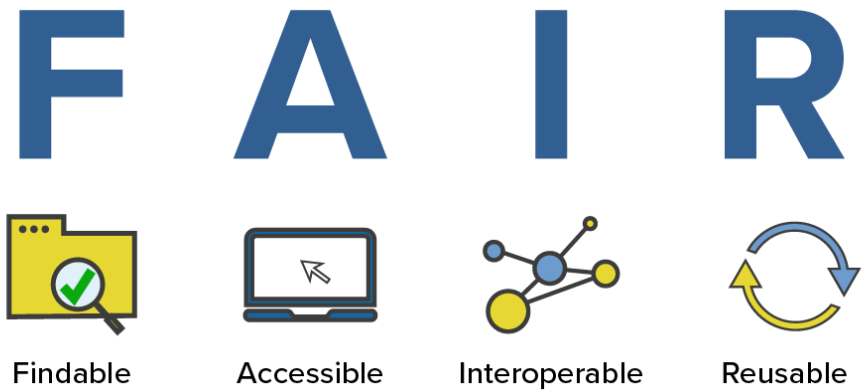
- Experimental design, protocol implementation and automation
- Experiment execution, ***data acquisition***
  - *Real-time sensor networks, IoT-enabled lab equipment, automated logging*
  - *High-speed imaging systems, continuous monitoring, continuous deployment*



Introduction

Why scientific computing?

- Experimental design, protocol implementation and automation
- Experiment execution, data acquisition
- Data ***storage***, curation and ***processing***
  - *Cloud HPC services growing 9.6% annually: exascale (10^{18}) computing & storage*
  - *FAIR compliance, automated metadata generation, distributed computing*



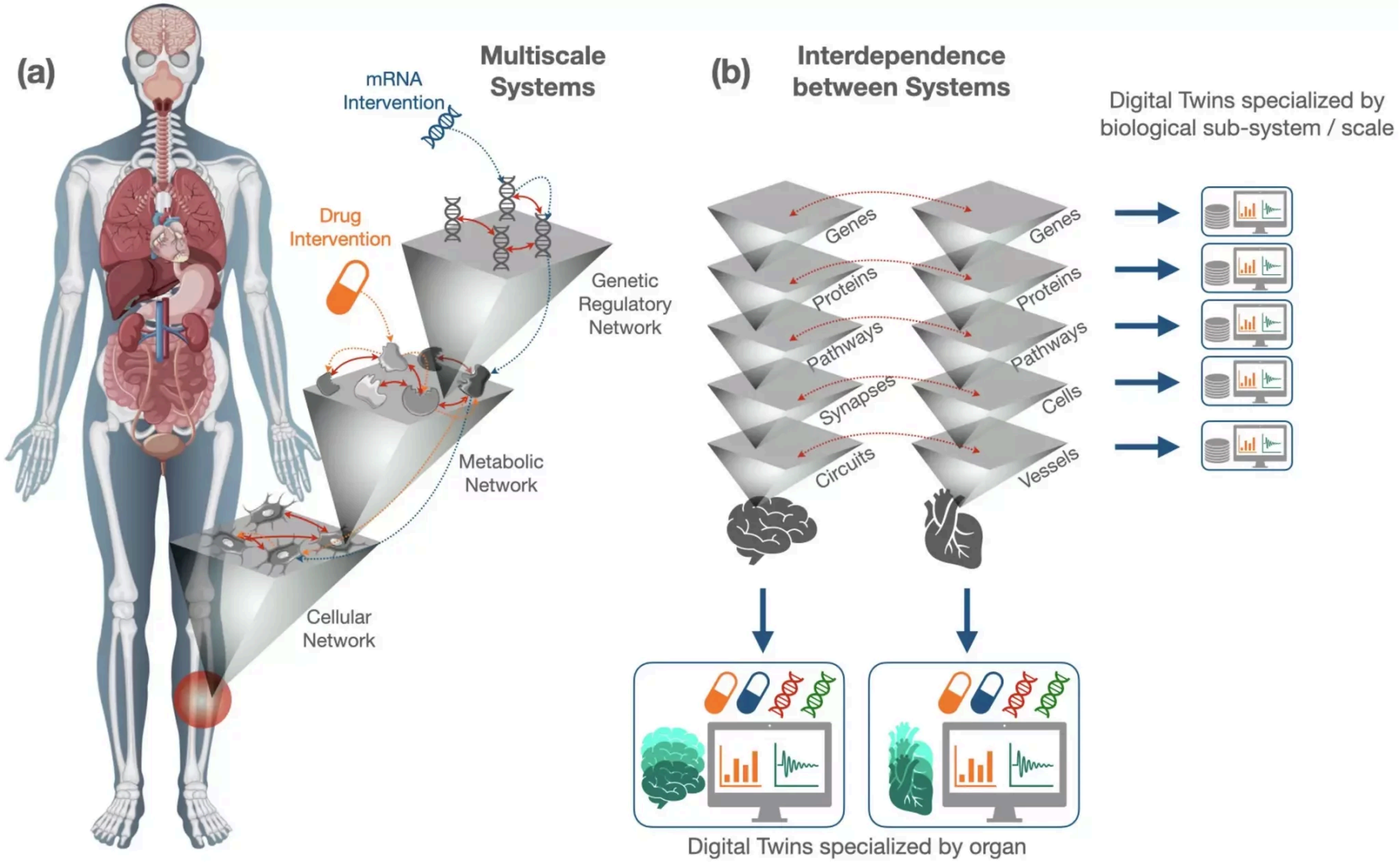


Introduction

- Experimental design, protocol implementation and automation
- Experiment execution, data acquisition
- Data storage, curation and processing
- Data ***analysis***, mining and ***visualization***
  - *Machine Learning pipelines, interactive dashboards, big data analytics, continuous monitoring, closed-loop experiments*

Introduction

Why scientific computing?



De Domenico, M., Allegri, L., Caldarelli, G. *et al. npj Digit. Med.* 8, 37 (2025). https://doi.org/10.1038/s41746-024-01402-3

Introduction

Why scientific computing?

Scientific process is increasingly **computationally intensive** (all domains):

- ***Experimental design***, protocol implementation and automation
- Experiment execution, ***data acquisition***
- Data ***storage***, curation and ***processing***
- Data ***analysis***, mining and visualization
- Numerical ***simulation*** and *in silico* hypothesis testing

⚙️ Control over every step of the process

Organization

Course Objectives

- Acquire **critical programming skills** and learn **good scientific programming practices** (hands-on)
- Understand the principles of ***reproducible research*** and the modern scientific development ecosystem
- Utilize **AI-assisted coding tools** effectively while maintaining a strong grasp of fundamental programming concepts
- Set up and manage a **scientific development environment** and workflows
- Master ***numerical computing*** with NumPy, ***basic plotting*** with Matplotlib, and ***basic analytics*** with SciPy, Pandas and Scikit-learn
- Understand error handling, debugging strategies, type hints, and code documentation for **robust code development**
- Apply **statistical analyses** and sophisticated **machine learning techniques** using Scikit-learn and understand the use and deployment of deep learning models with PyTorch and/or Tensorflow
- Develop simple **numerical simulations** of mathematical models of biological systems
- Develop sophisticated **data analysis** routines for (neuro)biological datasets
- Understand testing, continuous integration, code review, packaging, and distribution techniques for professional software development

Organization

Semester course: Friday afternoons

- 14 sessions: 11 lectures with hands-on practicals (1h+2/3h blocks), 2 project work sessions and 1 presentation session
- Home work (practice makes perfect, recommended)
- Evaluation: *Final project*
  - Apply to your own use-cases

Lecturer and Tutor:

- Renato Duarte (renato.duarte@cnc.uc.pt)

Course Materials: https://github.com/CNNC-Lab/intscipro

Introduction & Outlook

Syllabus & Callendar

Course Calendar:

1. Introduction & Modern Development Ecosystem
2. Programming Fundamentals
3. Development Tools & Workflow

Part I: *Fundamentals*

4. Numerical Computing Foundations
5. Data Manipulation & Analysis
6. Visualization & Communication
7. Intermediate Programming Concepts

Part II: *Scientific Computing Core*

8. Statistical Analysis & Machine Learning I
9. Statistical Analysis & Machine Learning II
10. Numerical Simulation & Modeling
11. Specialized Tools & Domain Applications
12. Professional Development & Advanced Topics

Part III: *Advanced Applications*

13. Student Projects
14. Student Presentations

Part I: *Fundamentals*

Day 1: Introduction & Modern Development Ecosystem

Lecture:

- What is scientific programming? Research programming vs. software development
- The development cycle & reproducible research principles
- Programming languages for science (Python, R, Julia comparison)
- **AI-assisted coding**: current status

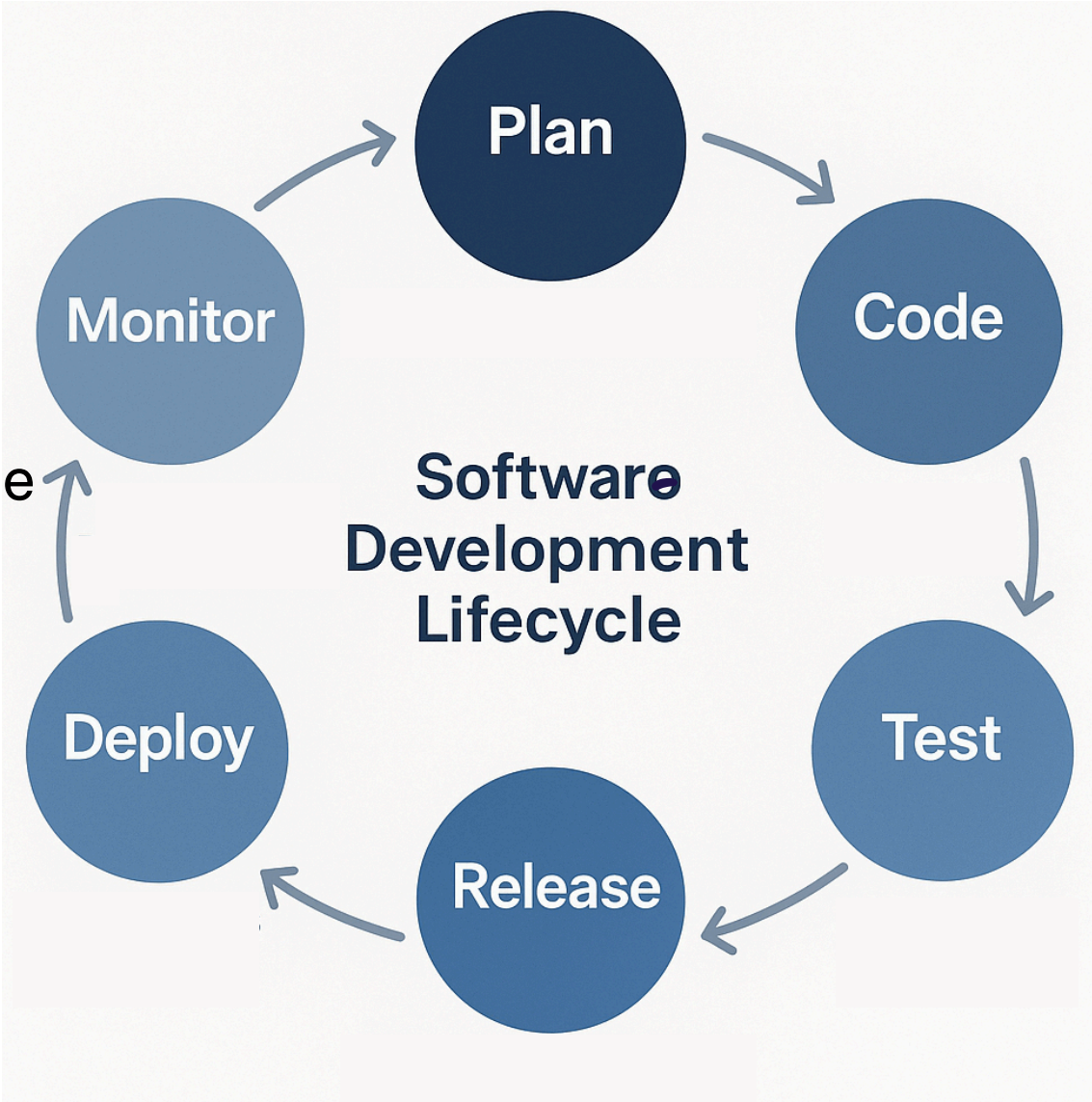
Practical:

- Setting up the development environment (IDEs, extensions, basic terminal, basic venv)
- AI-assisted coding demo: Chat-based, IDE integrated, agentic

The Development Cycle

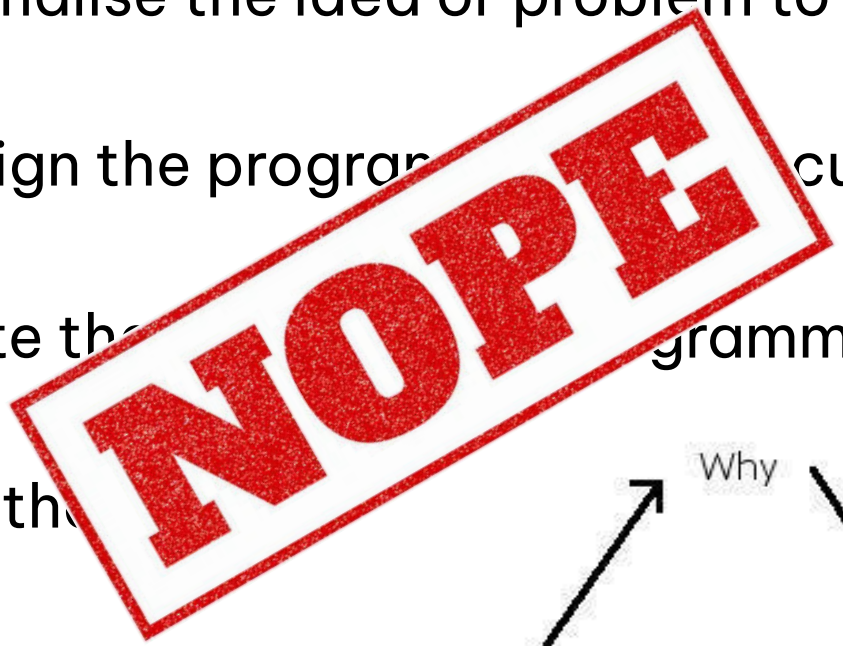
How does programming work?

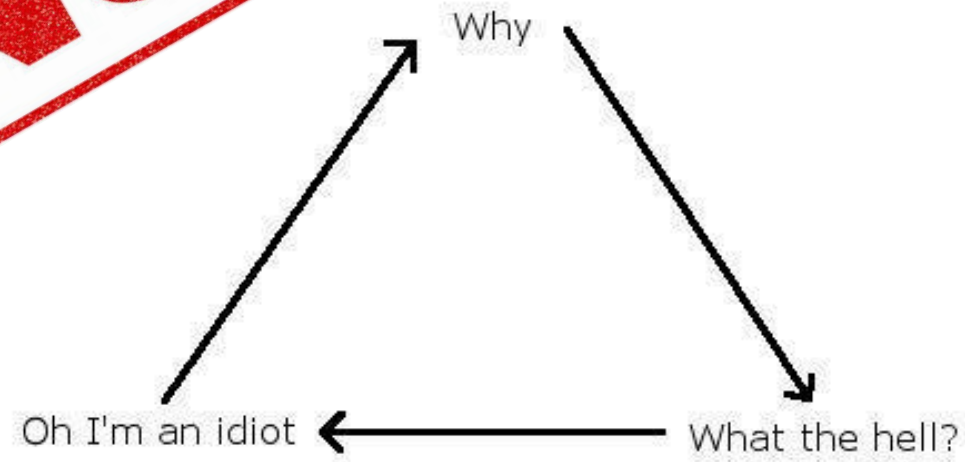
1. Formalise the idea or problem to be solved
2. Design the program flow to execute the solution
3. Write the program in a programming language
4. Run the program
5. Go home



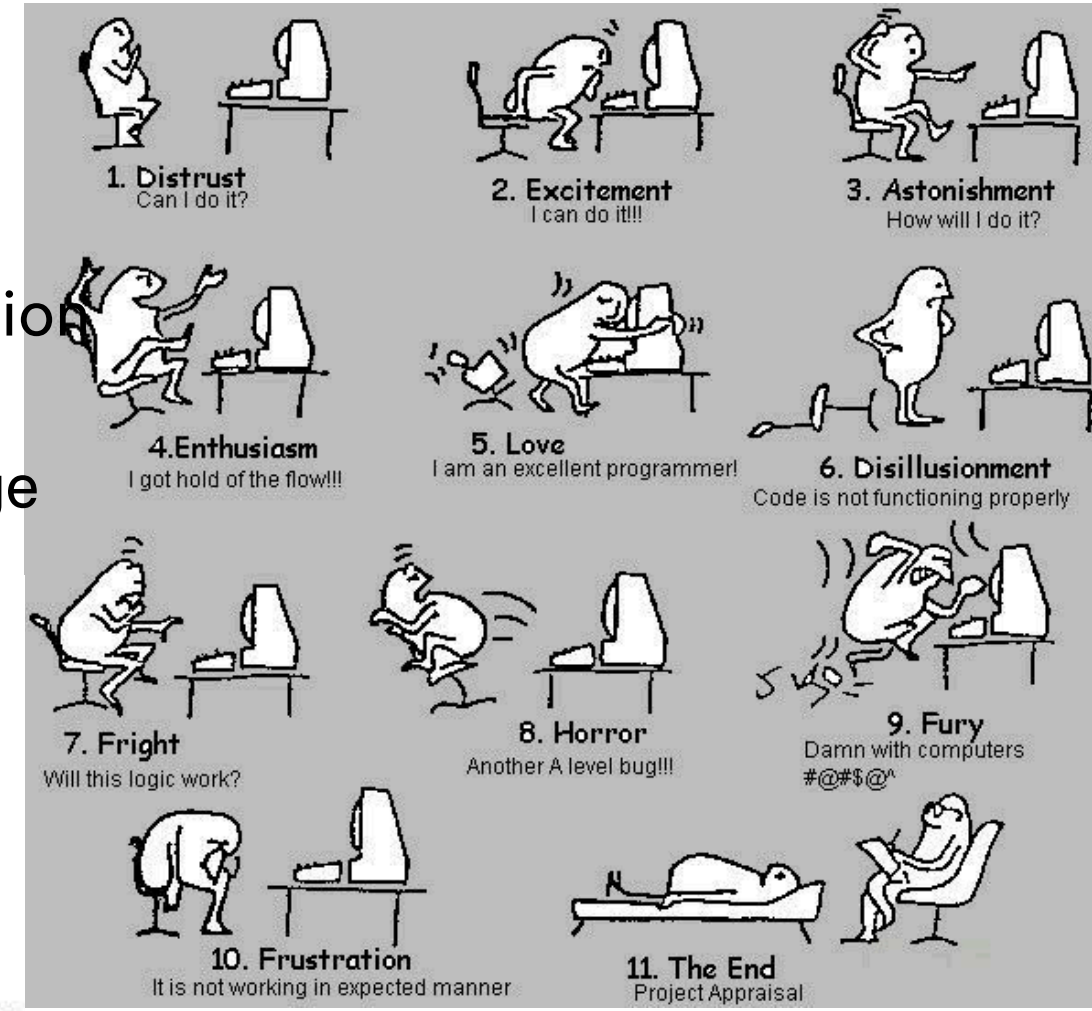
The Development Cycle

1. Formalise the idea or problem to be solved
2. Design the progra cute the solution
3. Write th gramming language
4. Run th
5. Go home





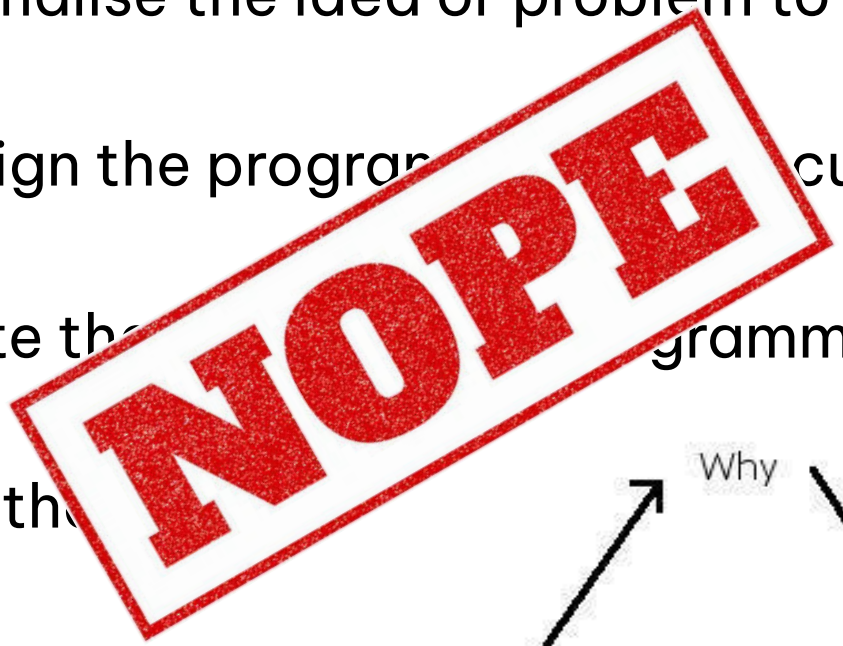
How does programming **really** work?

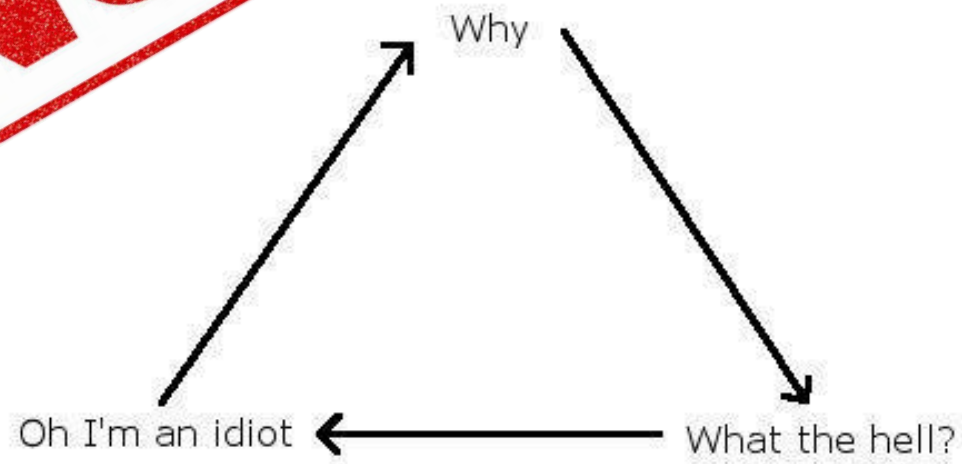


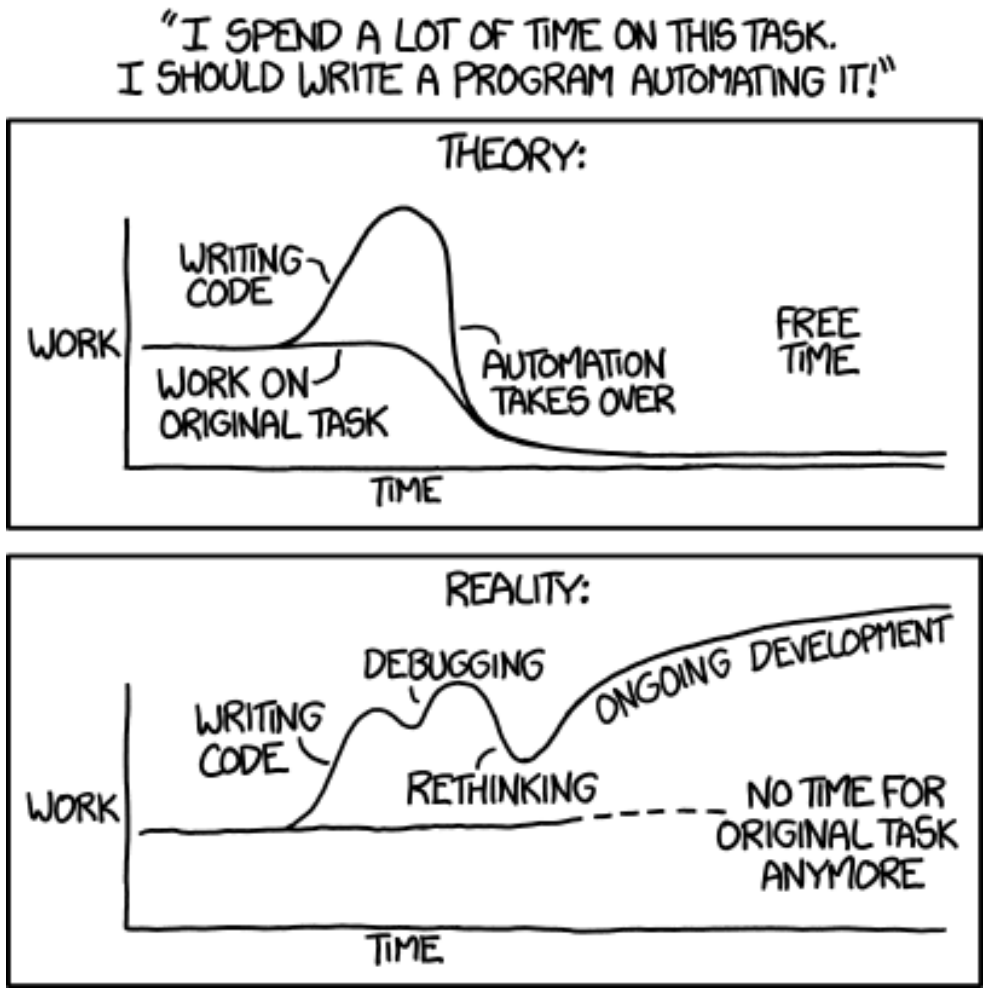
The Development Cycle

How does programming **really** work?

1. Formalise the idea or problem to be solved
2. Design the progra cute the solution
3. Write th gramming language
4. Run th
5. Go home

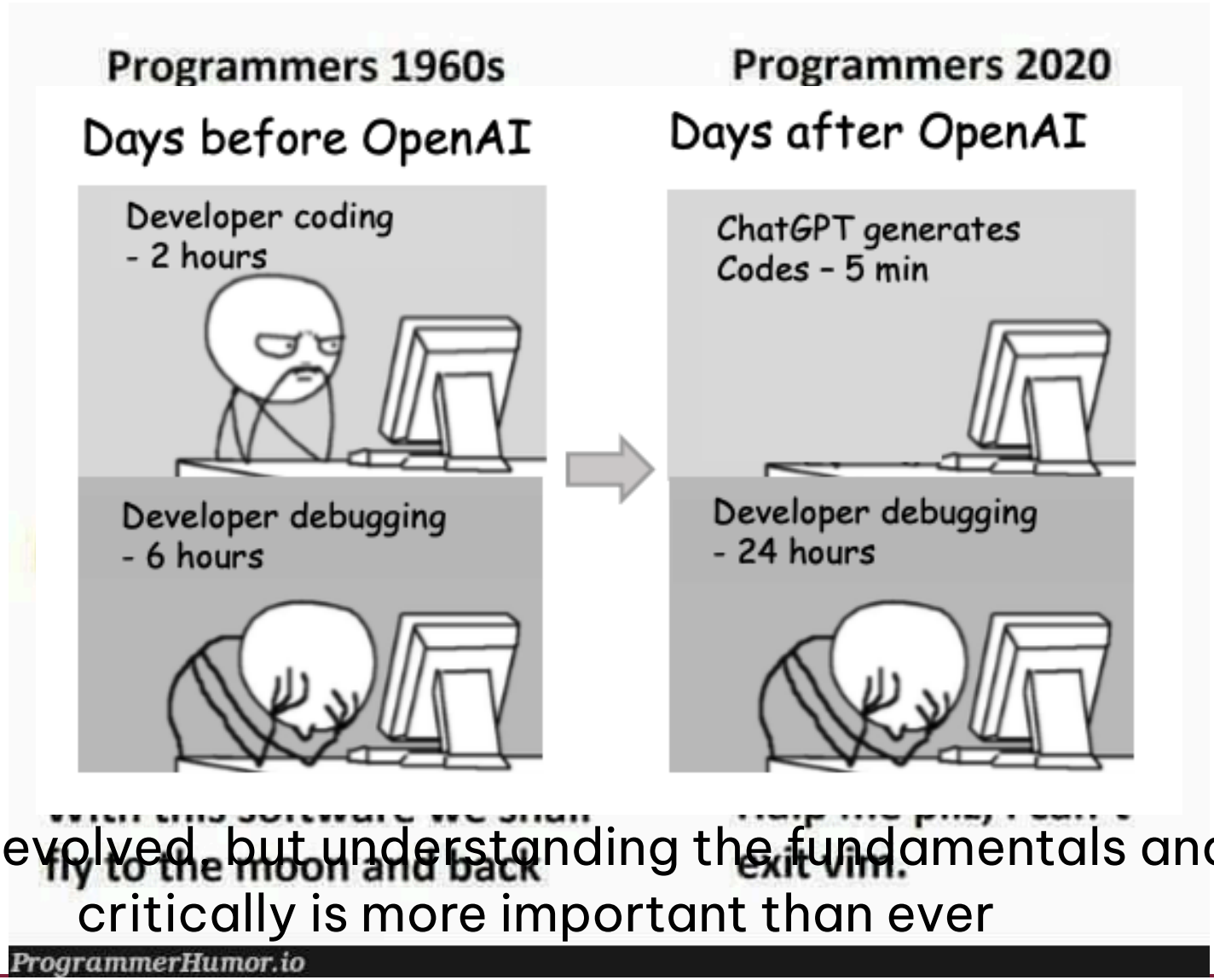






The Development Cycle

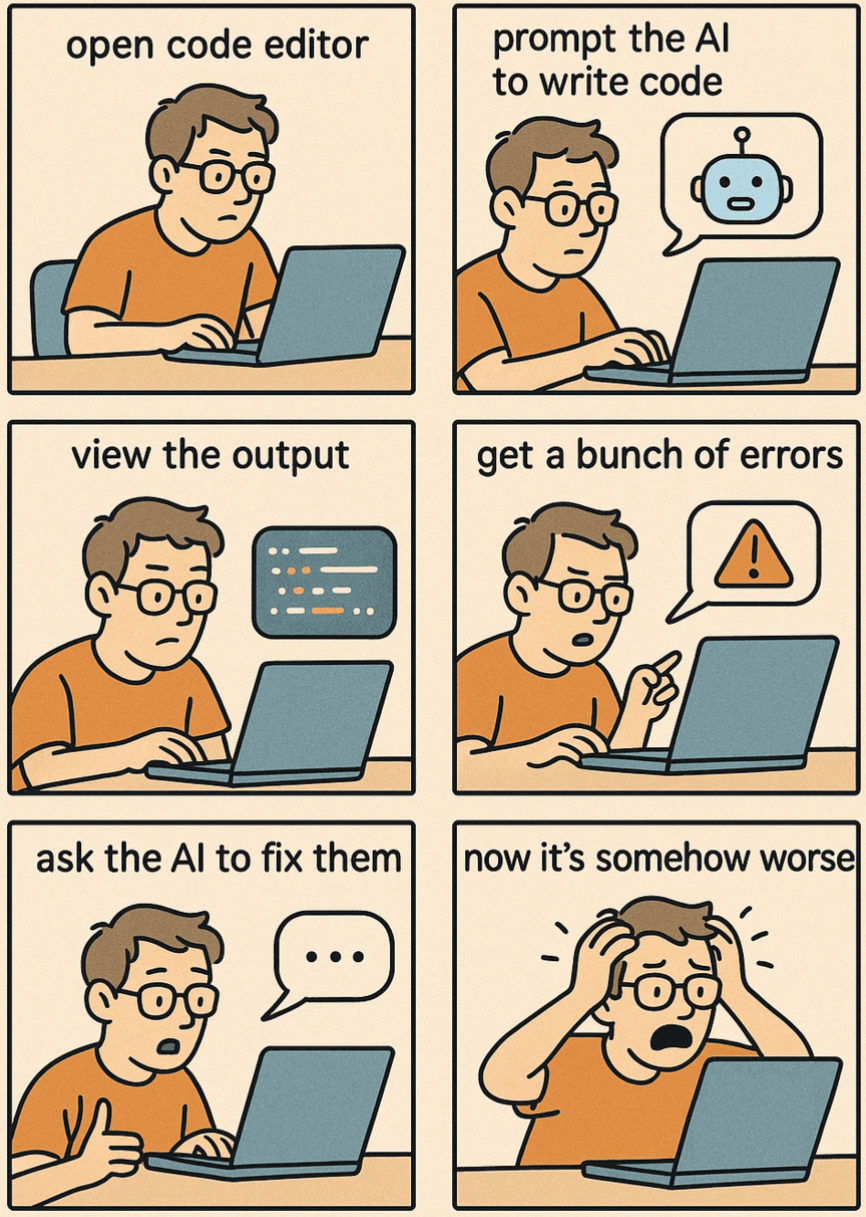
How does programming **really** work **today**?



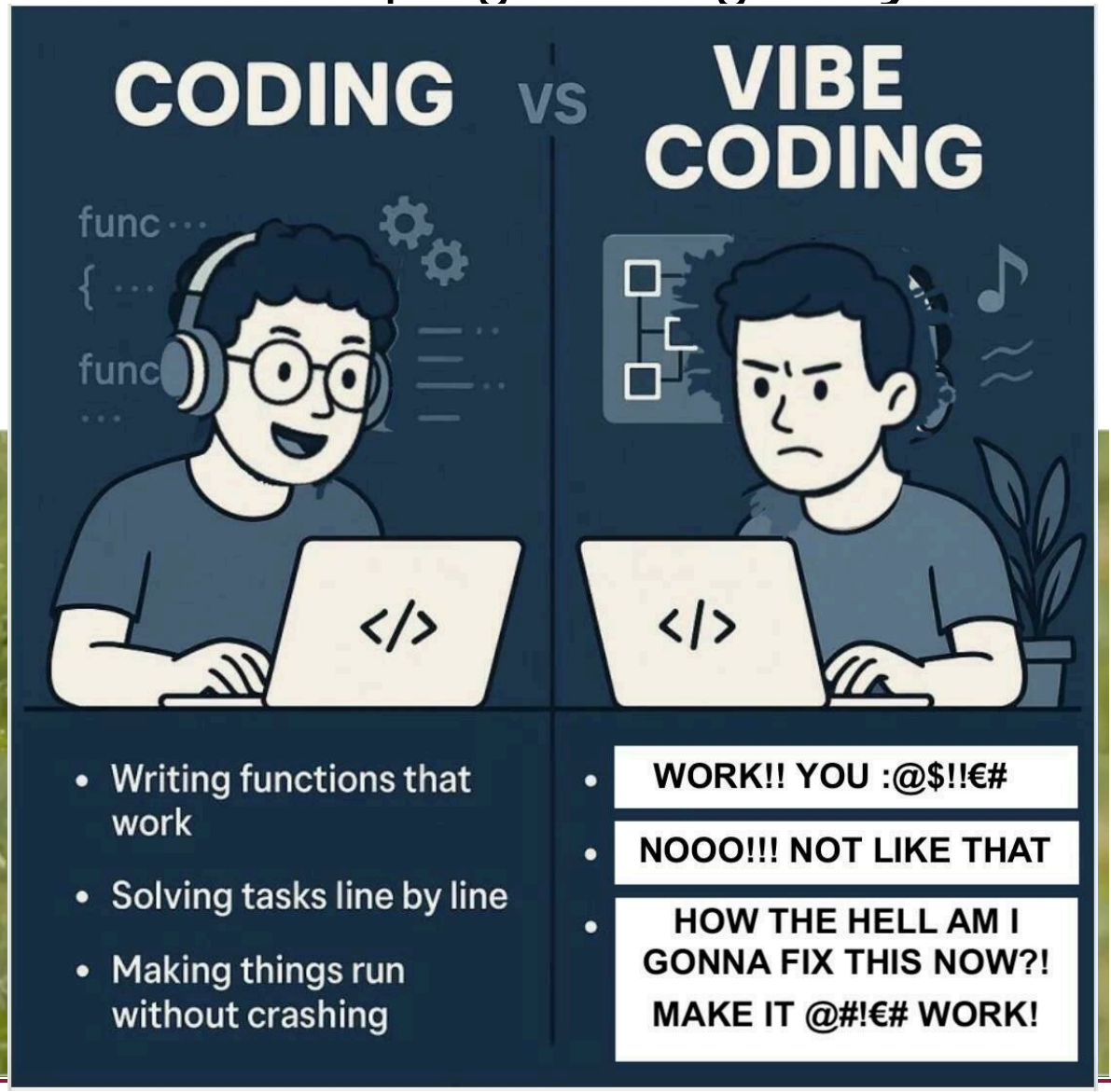
The tools have evolved, but understanding the fundamentals and thinking critically is more important than ever

The Development Cycle

how to be a vibe coder

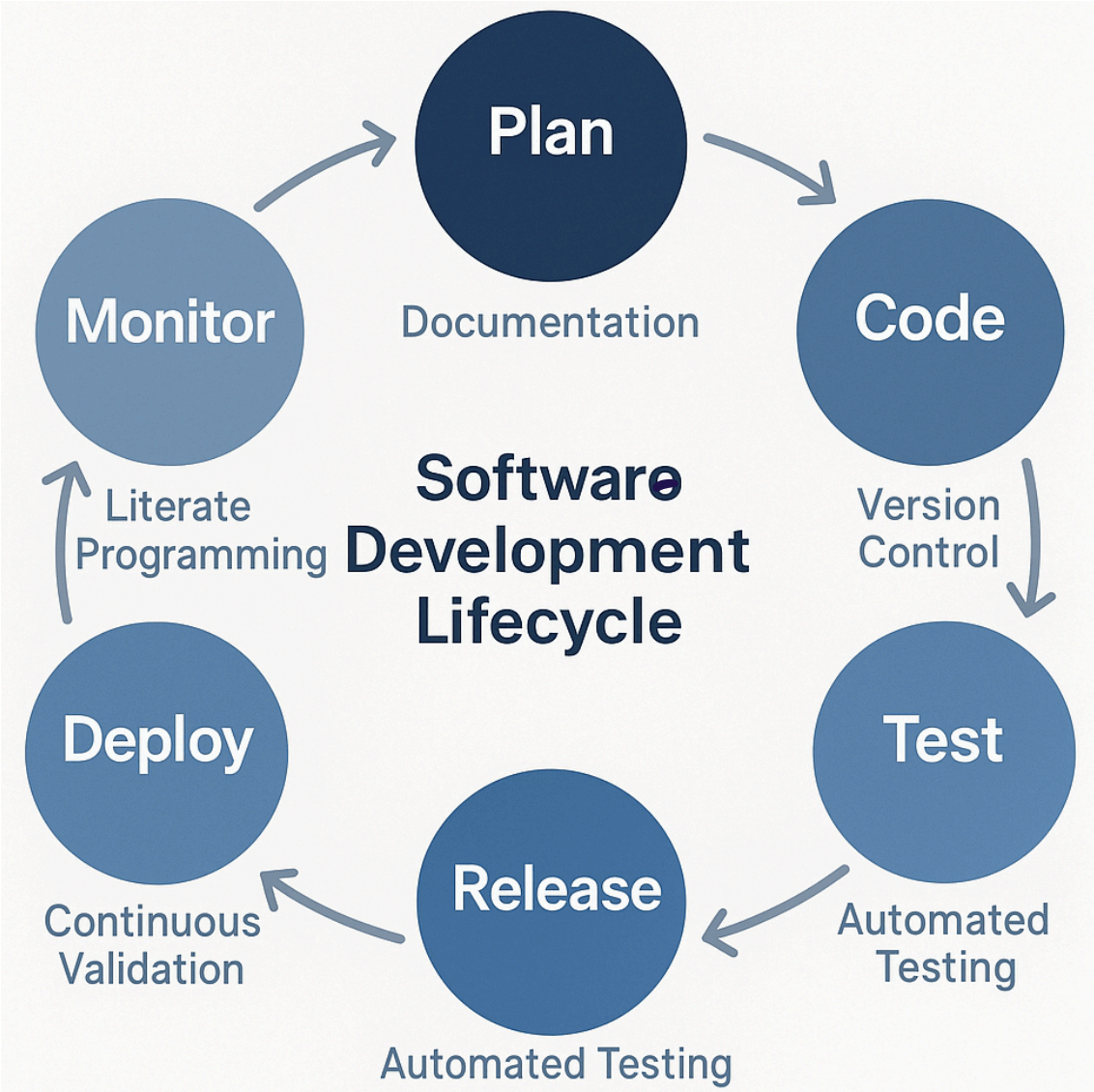


How does programming **really** work **today**?



The (reproducible) Development Cycle

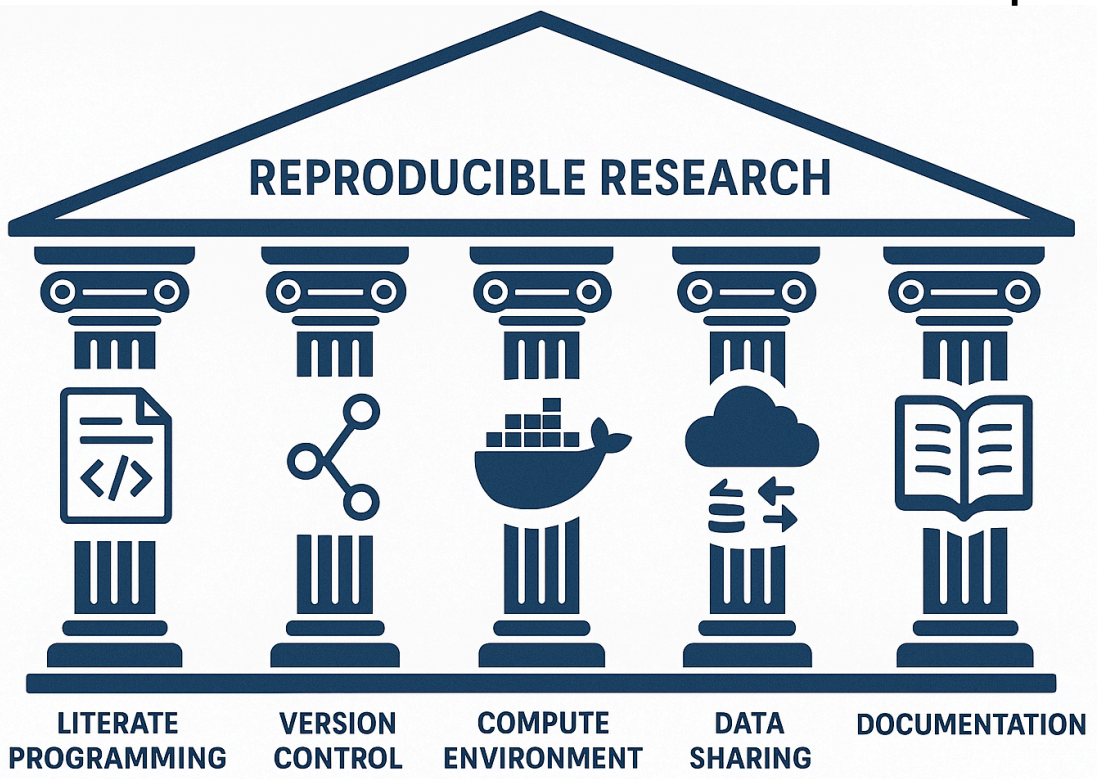
1. **Plan:** Documentation standards, data management plans
2. **Code:** Version control (Git), literate programming practices
3. **Build:** Containerization, environment management
4. **Test:** Automated validation, continuous integration (CI/CD)
5. **Release:** FAIR data sharing, code repositories
6. **Deploy:** Executable papers, reproducible workflows
7. **Monitor:** Continuous validation, maintenance



https://elifesciences.org/collections/d72819a9/executable-research-articles

The (reproducible) Development Cycle

Reproducibility & Replicability

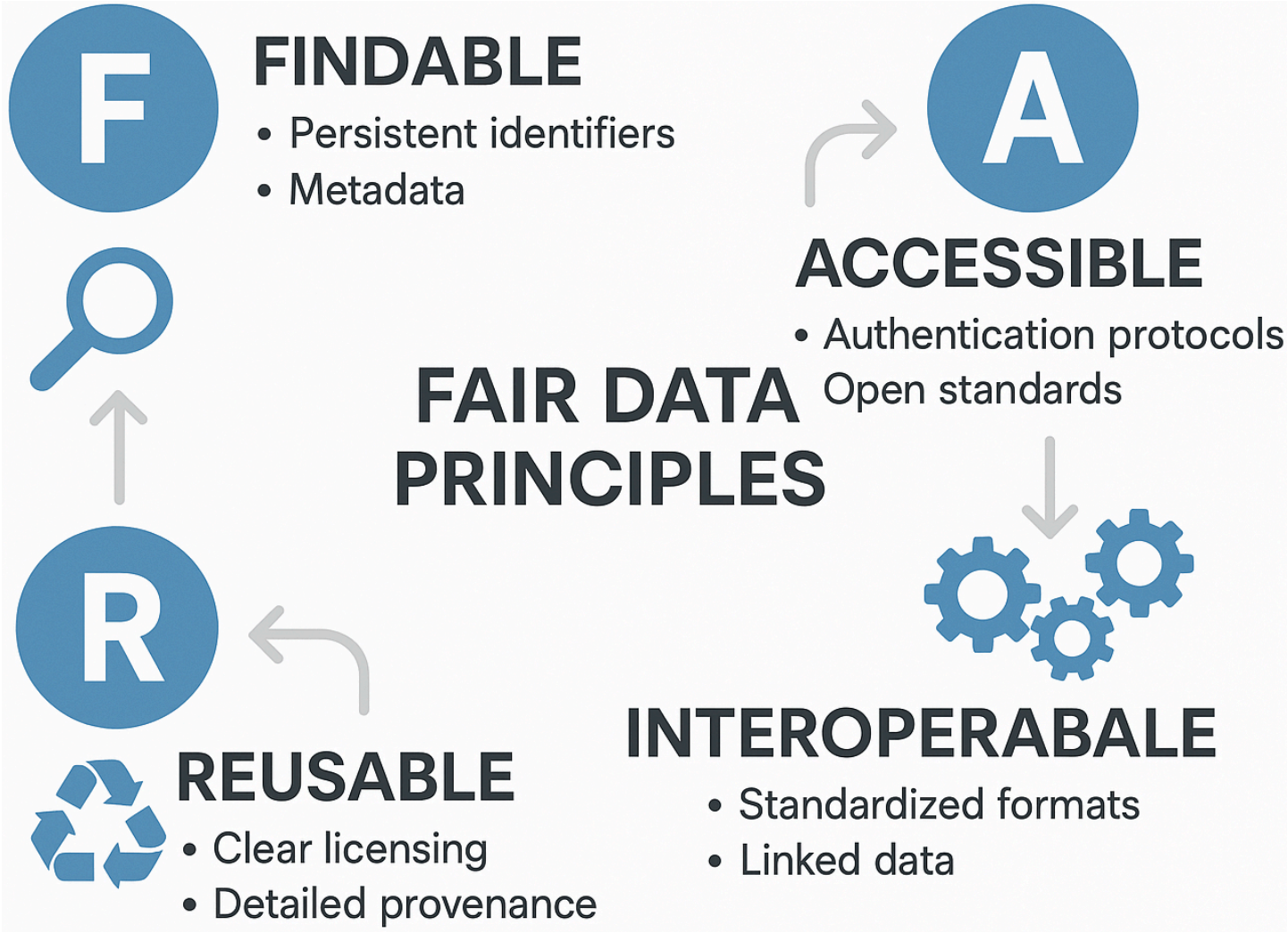


1. **Literate Programming**: Markdown, Jupyter notebooks: code + narrative + results
2. **Version Control**: Git workflows, GitHub/GitLab, persistent code sharing
3. **Compute Environment**: Containers (Docker), package management (Conda), reproducible builds
4. **Data Sharing**: FAIR principles, persistent repositories, linked data/code
5. **Documentation**: Detailed methods, README files, metadata standards

Sandve GK, Nekrutenko A, Taylor J, Hovig E (2013) Ten Simple Rules for Reproducible Computational Research. *PLoS Comput Biol* 9(10): e1003285.

The (reproducible) Development Cycle

FAIR data principles



Foundation for Reproducible Research

- **Findable**: Persistent identifiers, rich metadata, searchable resources
- **Accessible**: Standard protocols, authentication when needed, metadata persistence
- **Interoperable**: Standard formats, shared vocabularies, qualified references
- **Reusable**: Clear licensing, detailed provenance, community standards

The (reproducible) Development Cycle

⚠️ **Real-World Impact**: Duke Genomics Scandal (2006-2011)

Flawed bioinformatics analysis → Wrong patient treatments → Clinical trials terminated → Lawsuits → Retractions

Computational reproducibility is crucial for translating research into real-world outcomes

The (reproducible) Development Cycle

Reproducibility Crisis & Solutions

🚨 The Crisis

20% Computational studies reproducible

11% Bioinformatics articles reproduced

6.8% Data shared upon "reasonable request"

Key Barriers: Missing data • Broken dependencies • Poor documentation

💡 Modern Solutions

🐋 Containerization

Docker solves "works on my machine"

📓 Literate Programming

Code + narrative + results unified

🔄 FAIR Data

Machine-actionable principles

📊 Version Control

Git workflows enable collaboration

Selected results from published reproducibility surveys; exact values differ by field and study design.

Programming Languages

There are 3 main types of programming languages:

- ***Assembly languages*** are low-level languages that are used to talk to the hardware directly

```
; ------------------------------------------------------------------------------
; Writes "Hello, World" to the console using only system calls. Runs on 64-bit Linux only.
; To assemble and run:
;
;     nasm -felf64 hello.asm && ld hello.o && ./a.out
; ------------------------------------------------------------------------------

          global    _start

          section   .text
_start:   mov       rax, 1                  ; system call for write
          mov       rdi, 1                  ; file handle 1 is stdout
          mov       rsi, message            ; address of string to output
          mov       rdx, 13                 ; number of bytes
          syscall                           ; invoke operating system to do the write
          mov       rax, 60                 ; system call for exit
          xor       rdi, rdi                ; exit code 0
          syscall                           ; invoke operating system to exit

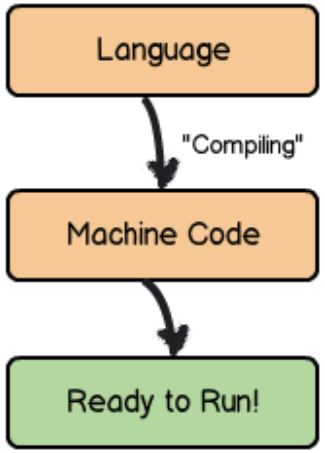
          section   .data
message:  db        "Hello, World", 10      ; note the newline at the end
```

Programming Languages

There are 3 main types of programming languages:

- ***Assembly languages*** are low-level languages that are used to talk to the hardware directly

- ***Compiled languages*** are intermediate languages where the program (or source code) is compiled into machine code and then executed by the hardware







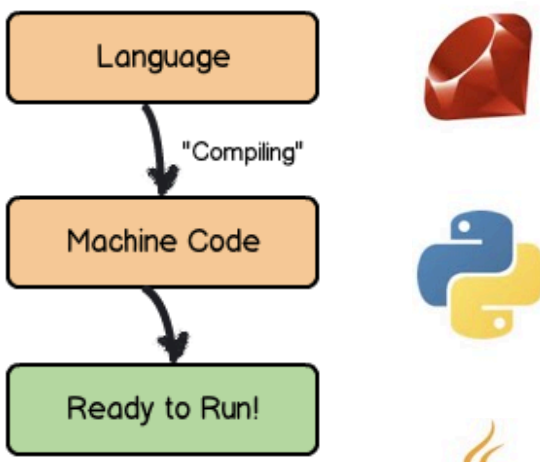


Programming Languages

There are 3 main types of programming languages:

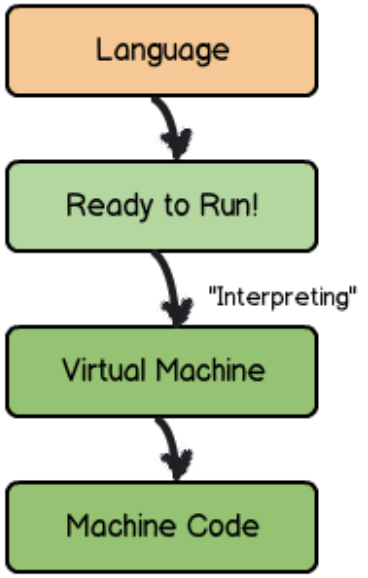
- ***Assembly languages*** are low-level languages that are used to talk to the hardware directly

- ***Compiled languages*** are intermediate languages where the program (or source code) is executed by the hardware





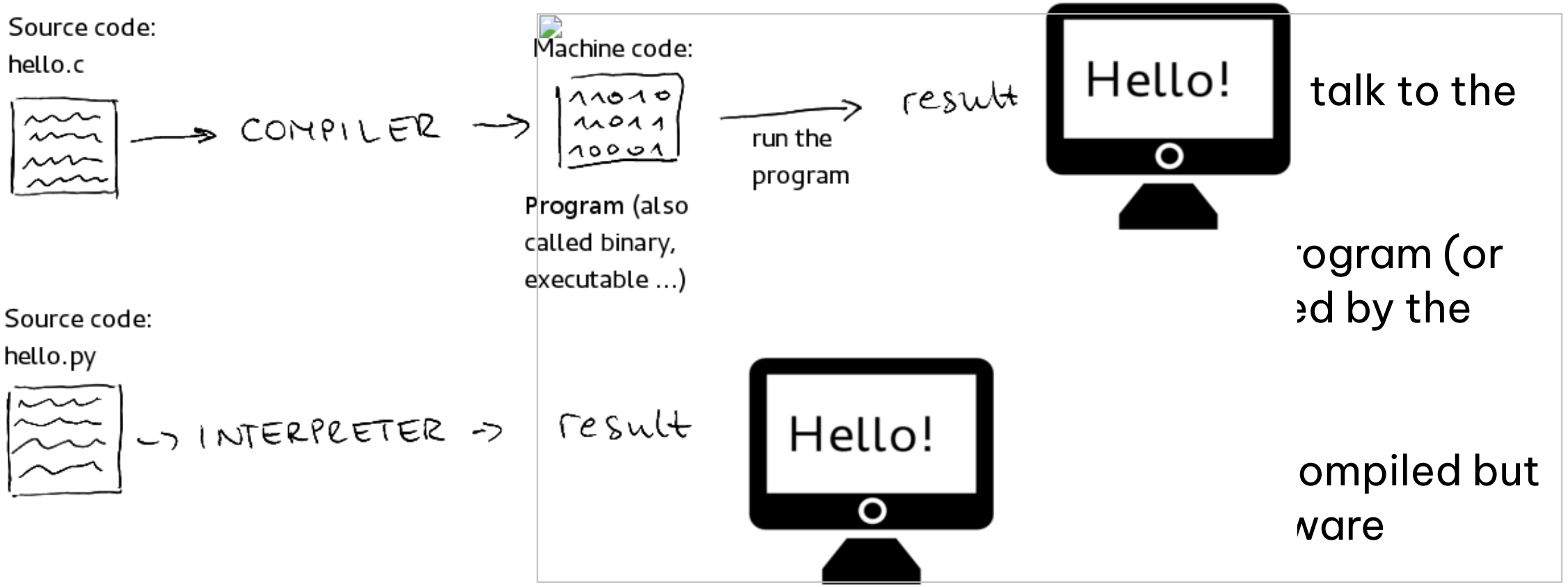




- ***Interpreted lang*** ıre not compiled but executed by an he hardware

Programming Languages

Why Python?



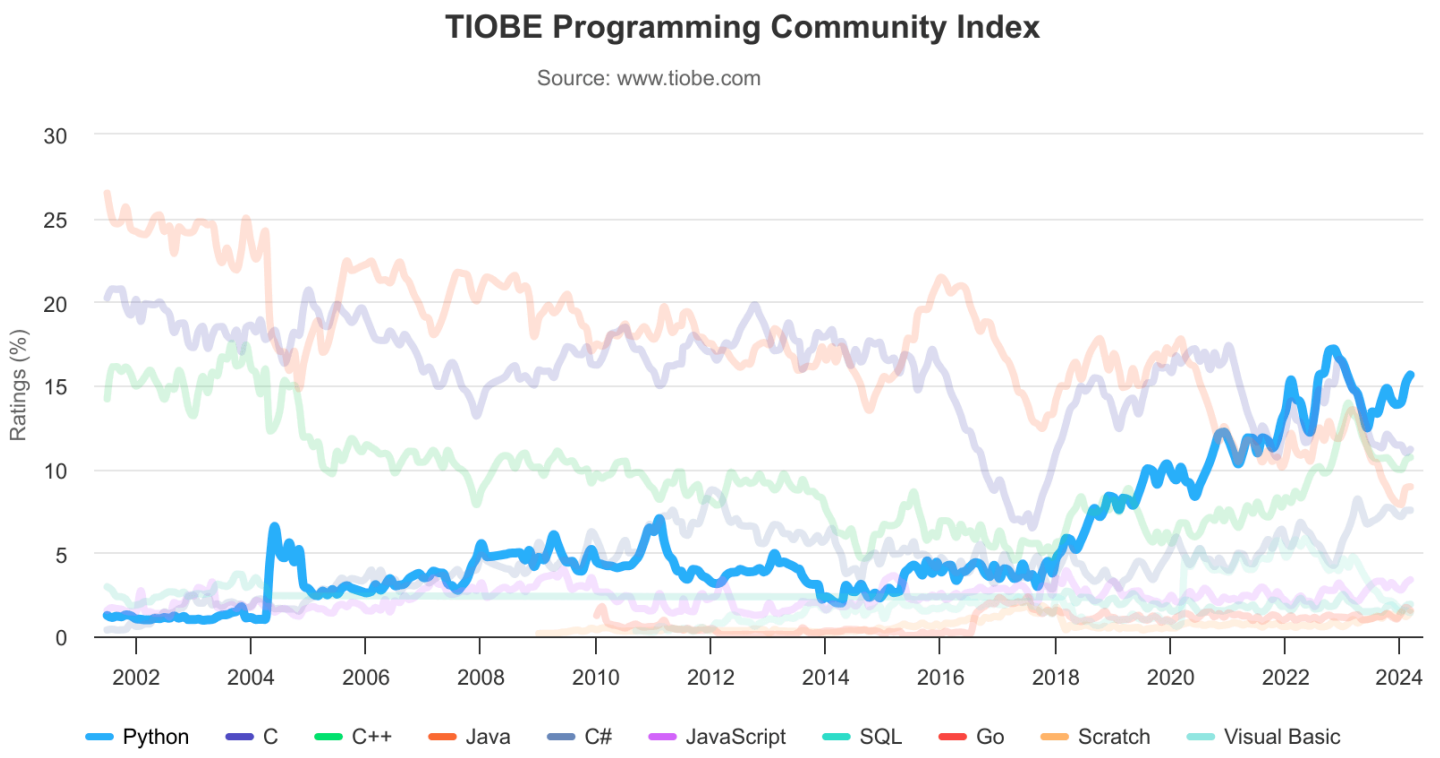
Python is an interpreted language

Programming Languages

Why Python?

- High-level, interpreted programming language
- Created in 1991 by Guido van Rossum
- Designed with emphasis on code readability and expressivity
- Supports multiple programming paradigms: procedural, object-oriented and functional

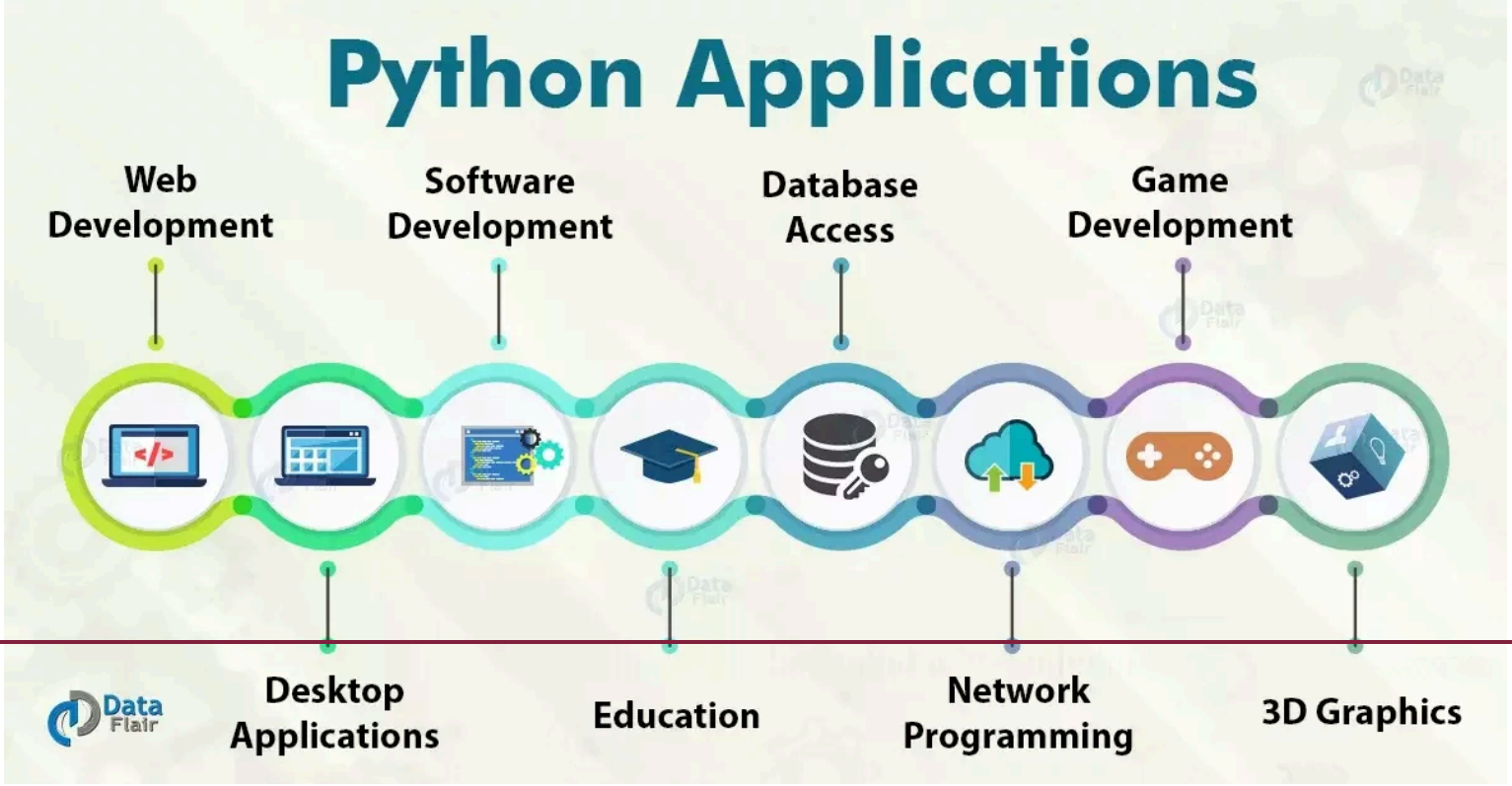
- The most popular among developers (particularly due to data science / ML)



Programming Languages

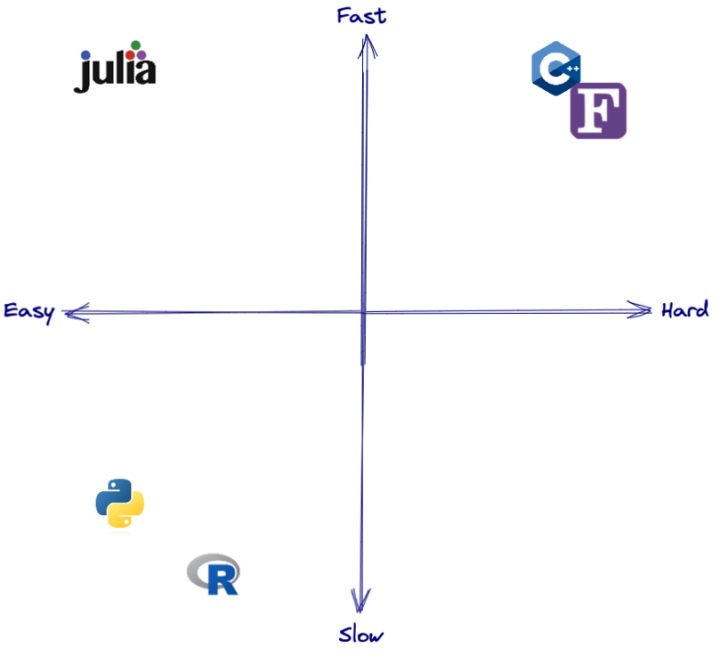
Why Python?

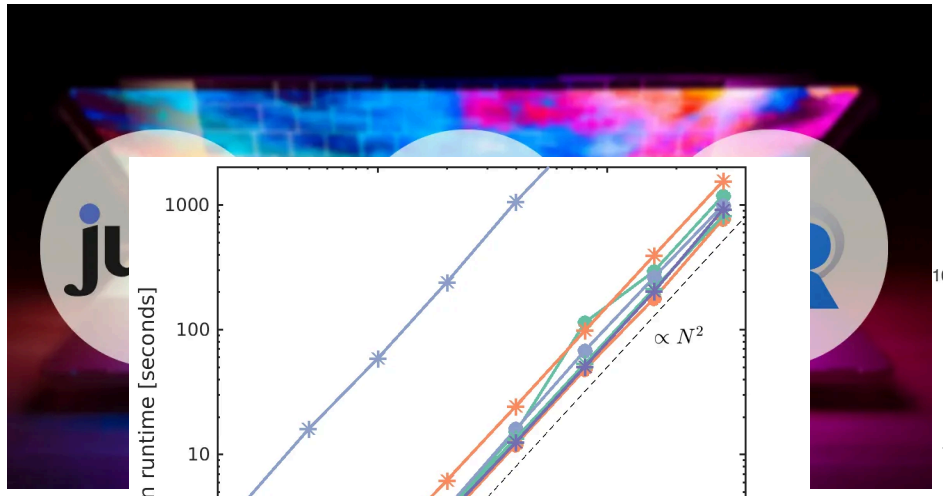
1. **Readability:** clear and human-readable syntax
2. **Easy to learn:** easy for beginners to pick up and understand
3. **Versatility:** not limited to a type of task; from web development to data science
4. **Rich library support:** both pre-included and community maintained
5. **Platform independence:** runs on any OS
6. **Interpreted language:** code is executed line-by-line, simplifies debugging
7. **Open source and free:** large community
8. **Dynamically typed:** no need to declare the type of a variable when it is created. Python interpreter can infer the type

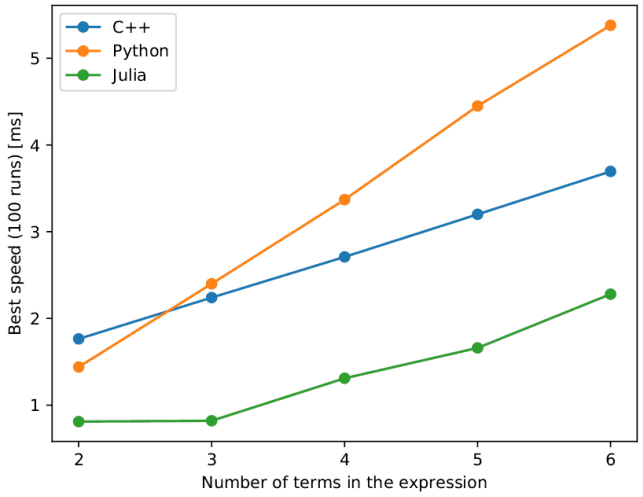


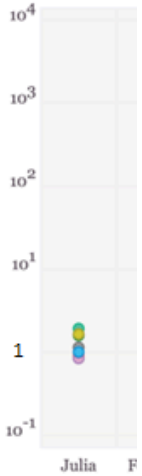
Programming Languages

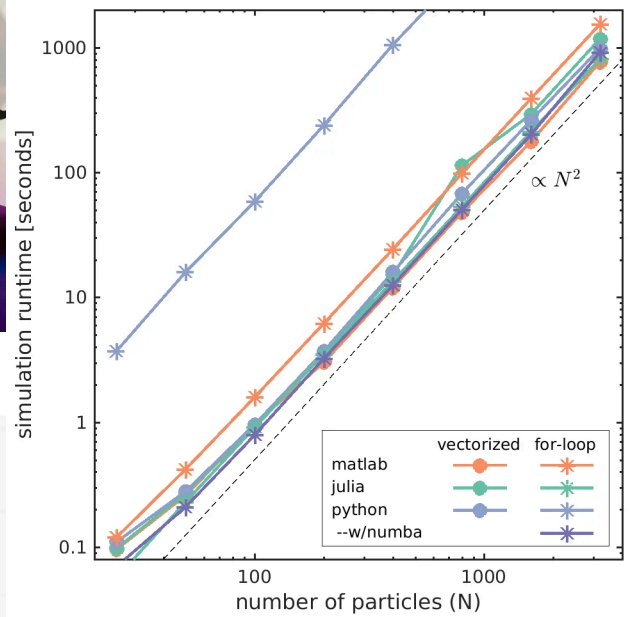
Why Python?

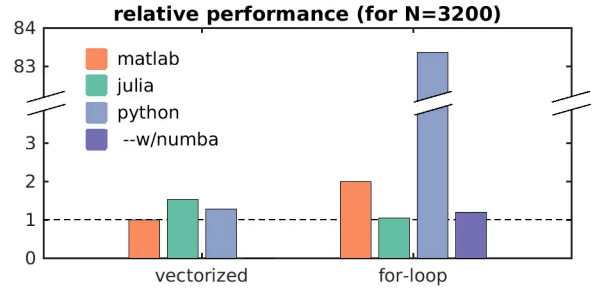




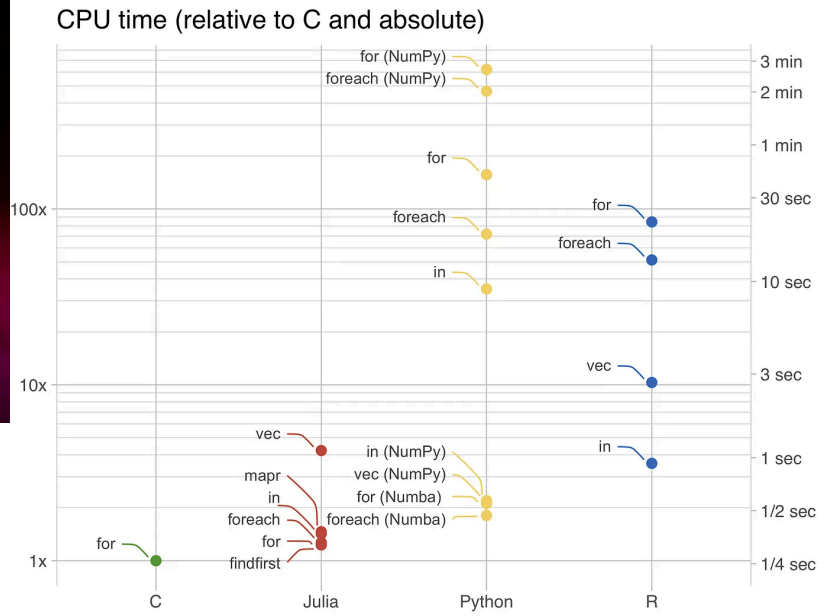


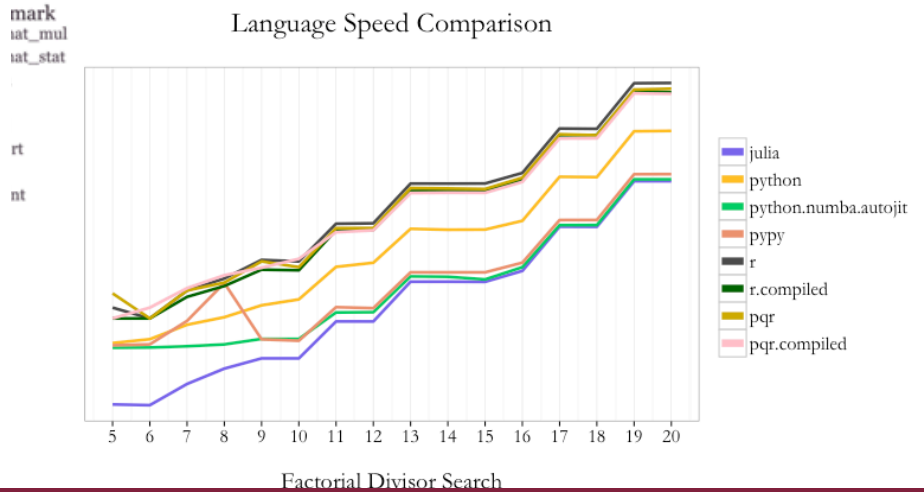






Python is SLOW...





Good Programming Practices

PEP8 and coding conventions

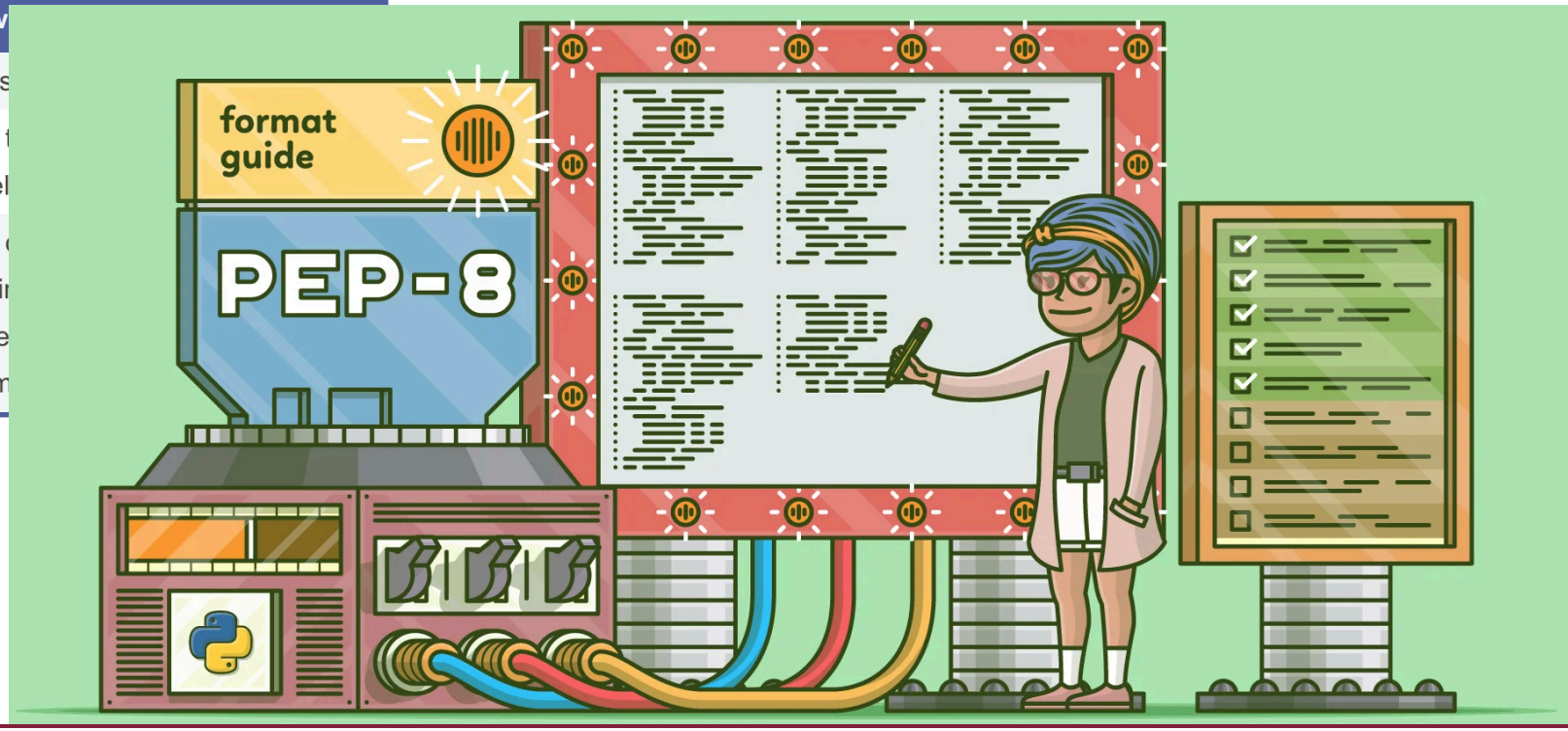
A good IDE will teach you

⚠️ Write **machine-actionable** code ⚠️

Naming conventions	
Never use l, O, or I single letter names as these can be mistaken for 1 and 0, depending on typeface	O = 2 # This may look like you're trying to reassign 2 to zero
Function	function, my_function
Variable	x, var, my_variable
Class	Model, MyClass
Method	class_method, method
Constant	CONSTANT, MY_CONSTANT, MY_LONG_CONSTANT
Module	module.py, my_mod-

Code Layout
Surround top-level functions and classes with two blank lines
Surround method definitions inside classes with a single blank line.
Use blank lines sparingly inside functions to show clear steps.

Indentation Follow
There are two styles
The first of these is t with the opening del
An alternative style break is a hanging i term meaning that e paragraph or statem



Good Programming Practices

1. **Write programs for people**, not computers.
    1. A program should not require its readers to hold more than a handful of facts in memory at once.
    2. Make **names consistent, distinctive, and meaningful**.
    3. Make code style and formatting consistent.
2. Let the computer do the work.
    1. Make the computer **repeat tasks**.
    2. Save recent commands in a file for re-use.
    3. Use a build tool to **automate workflows**.
3. Make **incremental changes**.
    1. Work in small steps with frequent feedback and course correction.
    2. **Use a version control system**.
    3. Put everything that has been created manually in version control.
4. **Don't repeat** yourself (or others).
    1. Every piece of data must have a single authoritative representation in the system.
    2. **Modularize code** rather than copying and pasting.
    3. Re-use code instead of rewriting it.
5. **Plan for mistakes**.
    1. Add assertions to programs to check their operation.
    2. Use an off-the-shelf unit testing library.
    3. Turn bugs into test cases.
    4. Use a symbolic debugger.
6. Optimize software only after it works correctly.
    1. Use a profiler to identify bottlenecks.
    2. **Write code in the highest-level language possible**.
7. **Document design and purpose**, not mechanics.
    1. Document interfaces and reasons, not implementations.
    2. Refactor code in preference to explaining how it works.
    3. Embed the documentation for a piece of software in that software.
8. Collaborate.
    1. Use pre-merge code reviews.
    2. Use pair programming when bringing someone new up to speed and when tackling particularly tricky problems.
    3. Use an **issue tracking tool**.

Wilson G, Aruliah DA, Brown CT, Chue Hong NP, Davis M, Guy RT, et al. (2014) Best Practices for Scientific Computing. PLoS Biol 12(1): e1001745. https://doi.org/10.1371/journal.pbio.1001745

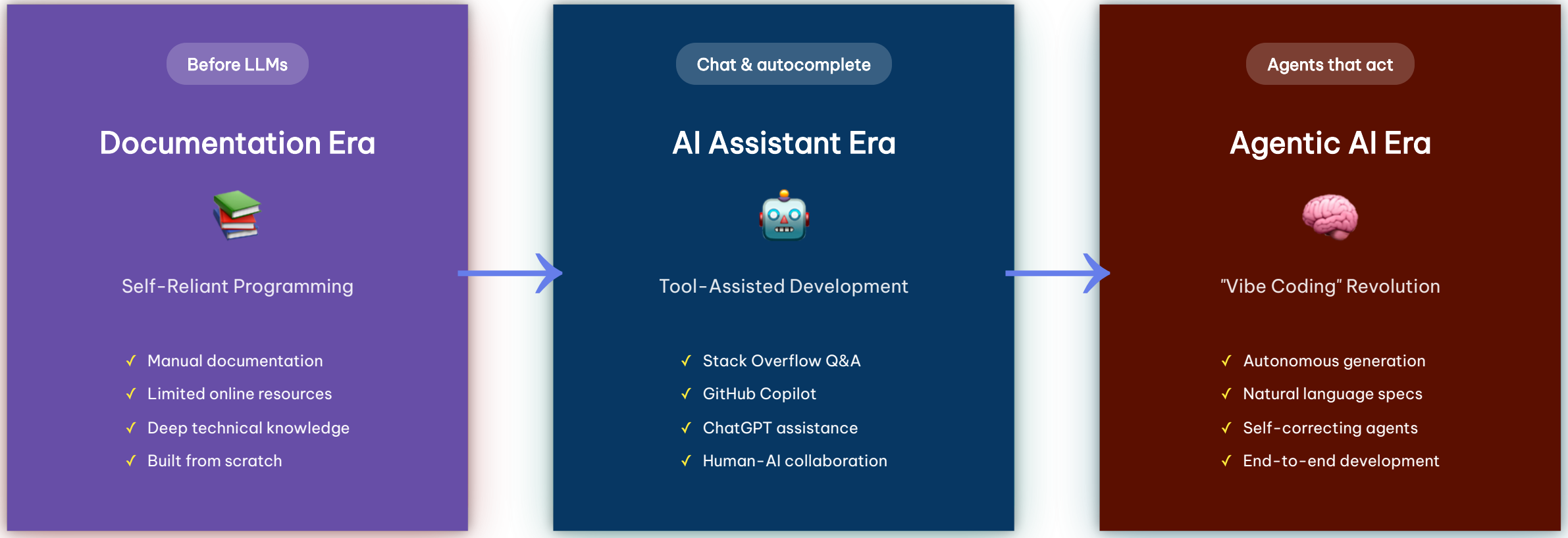
What about AI?

Is programming obsolete?

> *“ It’s going to sound completely opposite of what people feel. You probably recall over the course of the last 10–15 years, almost everybody would tell you it is vital that your children learn computer science… everybody should learn how to program. In fact, it’s almost exactly the opposite. It is our job to create computing technology such that nobody has to program and that the programming language is human.* ***Everybody in the world is now a programmer****. This is the miracle of artificial intelligence*
>
> Jensen Huang, nVIDIA’s CEO

AI-assisted coding

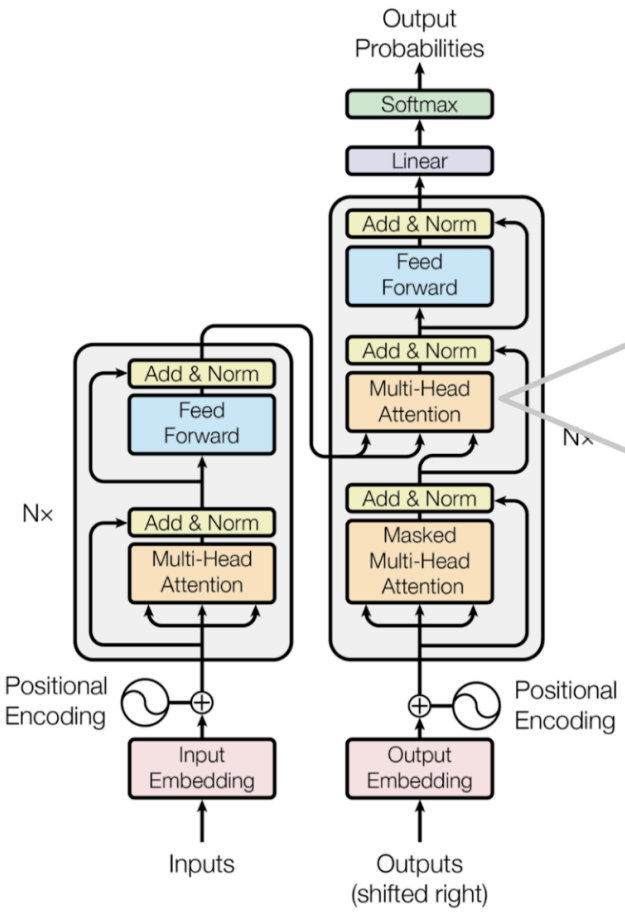
The evolution of programming assistance

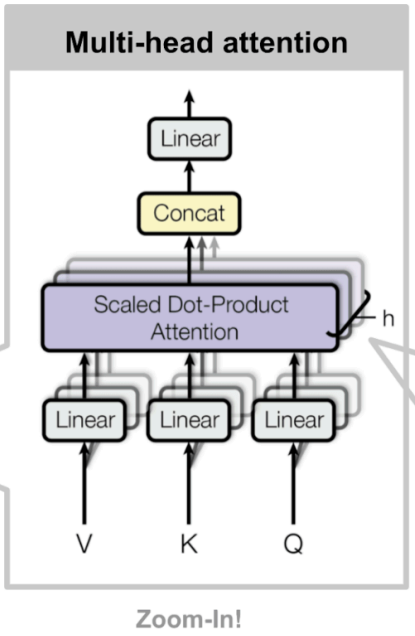


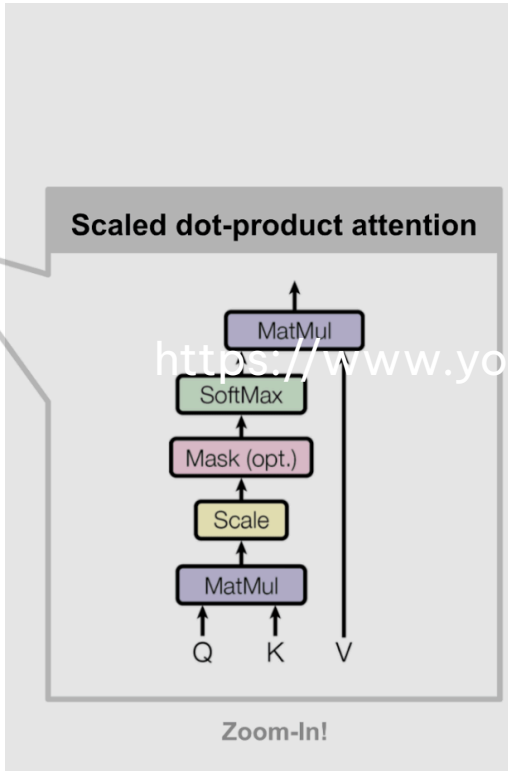
More delegation, more verification: AI lowers the cost of writing code, not the cost of being wrong

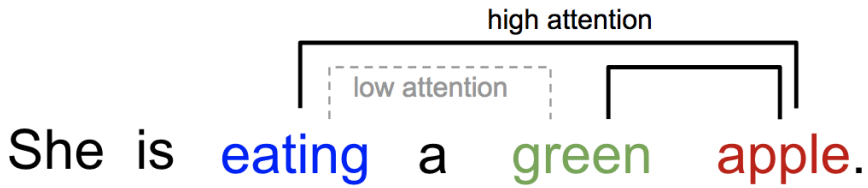
AI-assisted coding

What is a LLM?









AI-assisted coding

Current landscape



Chat

You paste, it answers

ChatGPT
Claude
Gemini

Learn, explain, draft, debug.
You run and check the code.



In-editor assistants

It suggests while you type

GitHub Copilot
Cursor

Autocomplete and edits inside your IDE.
You stay in control of every line.



Agents that act

It reads, edits and runs

Claude Code

today's live demo

other coding agents, cloud agents,
web app builders (v0, Bolt.new, Replit)

Plans, writes files, runs code, fixes errors.
You direct and verify.

Key trend: from assistance to autonomy

The human's job moves from typing code to directing, reviewing and verifying it

AI-assisted coding

Delegate, verify, own

1 · Delegate

- Boilerplate and first drafts
- Plots and formatting
- Refactoring and tests
- Explaining unfamiliar code

Delegate what is cheap to check.

2 · Verify

- Run it on a case with a known answer
- Read every line you cannot explain
- Ask for its assumptions
- Test edge cases
- Check numbers against the raw data
- Keep prompts and versions with the code

3 · Own

- You are accountable for the result
- Understand it well enough to defend it
- Document how AI was used
- Keep practising the fundamentals

Whoever wrote the code, you signed the paper.

AI lowers the cost of writing code, not the cost of being wrong

AI-assisted coding

What can go wrong

Invented things

Functions, packages, options and citations that do not exist

Plausible but wrong statistics

Code that runs and gives a number, with the wrong test or unit of analysis

Data privacy

Never paste patient or unpublished data into a tool you have not cleared

Licensing and attribution

Know where code came from and what you may reuse

Skills you stop practising

If you cannot review it, you cannot trust it

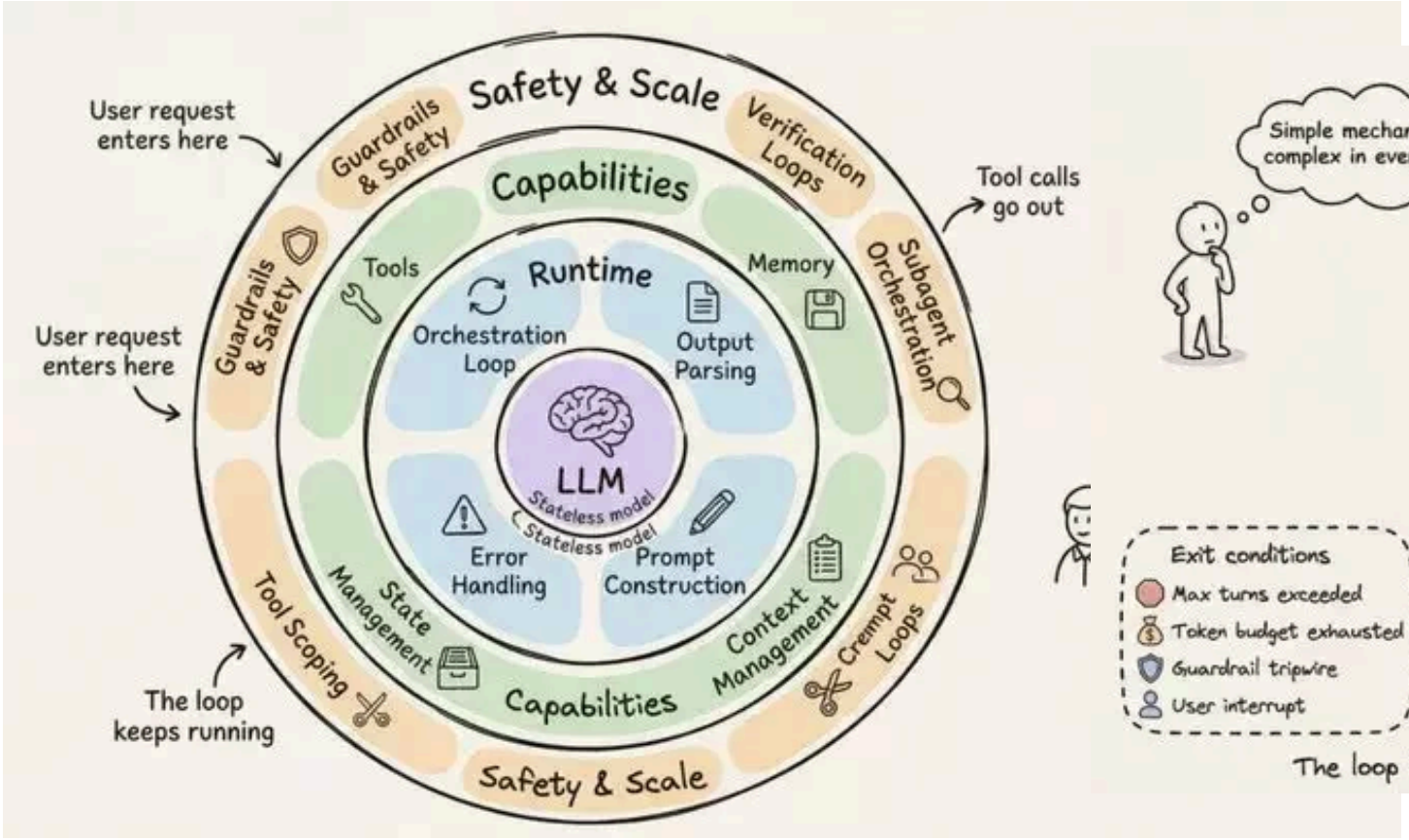
Irreproducible workflows

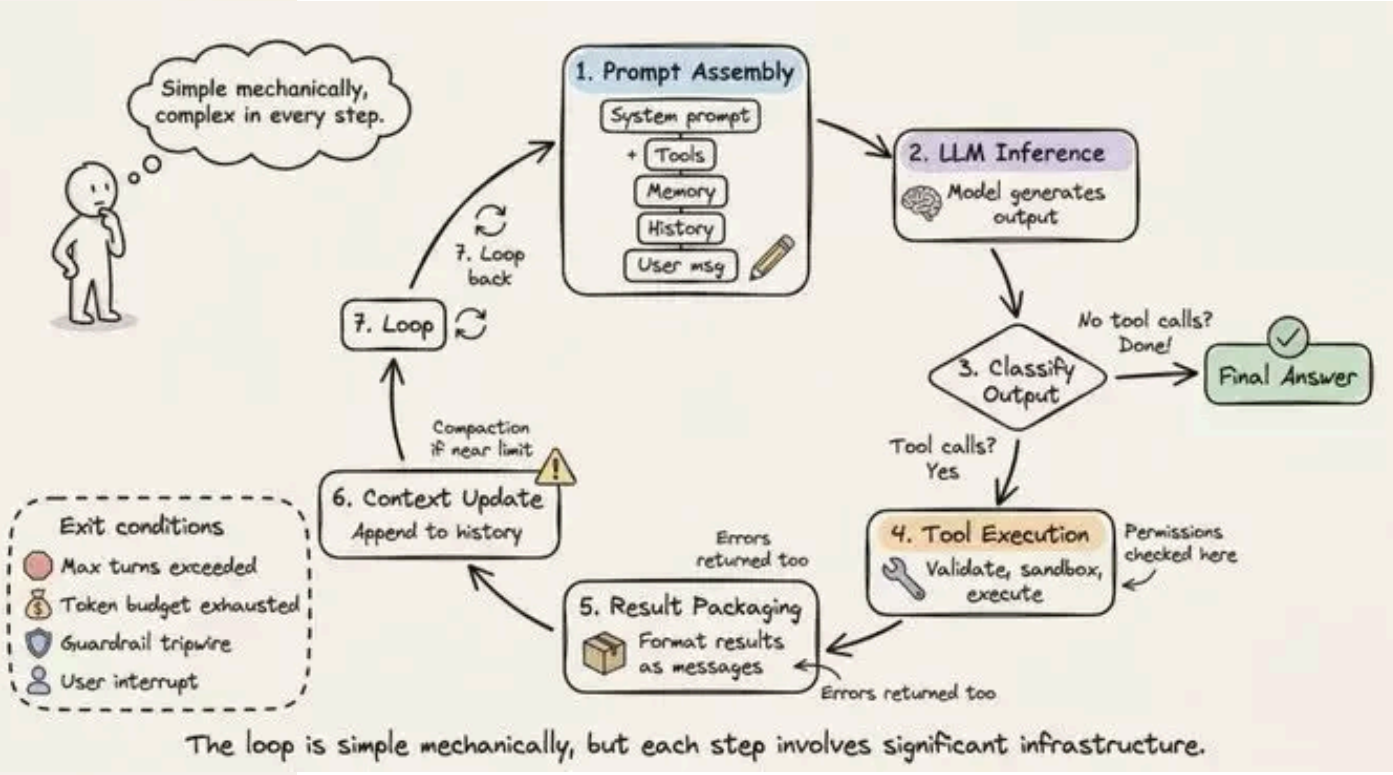
Unrecorded prompts, tool versions and manual edits

AI lowers the cost of writing code, not the cost of being wrong

AI-assisted research (life)

AI as a super-power

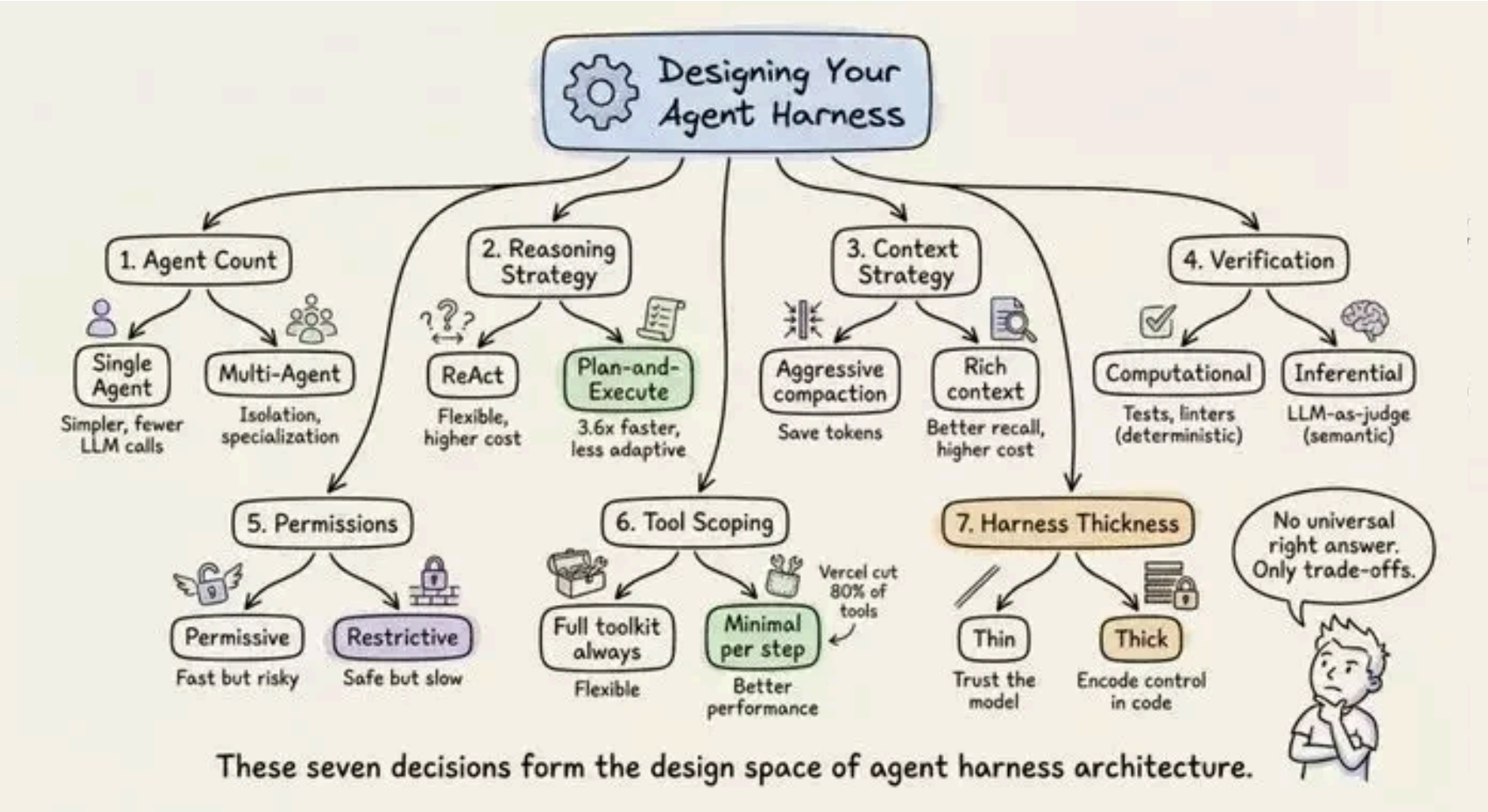




https://blog.dailydoseofds.com/p/the-anatomy-of-an-agent-harness

AI-assisted research (life)

AI as a super-power



https://blog.dailydoseofds.com/p/the-anatomy-of-an-agent-harness

AI-assisted research (life)

AI as a super-power

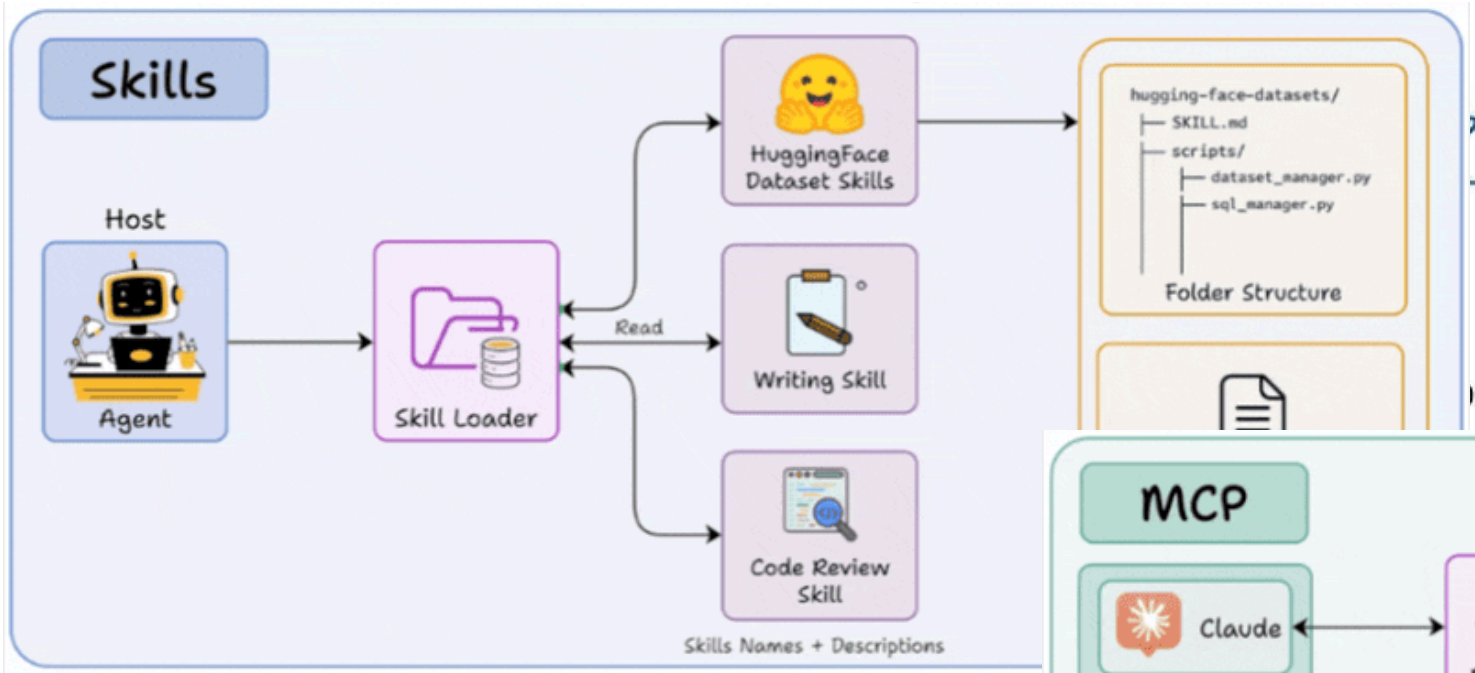




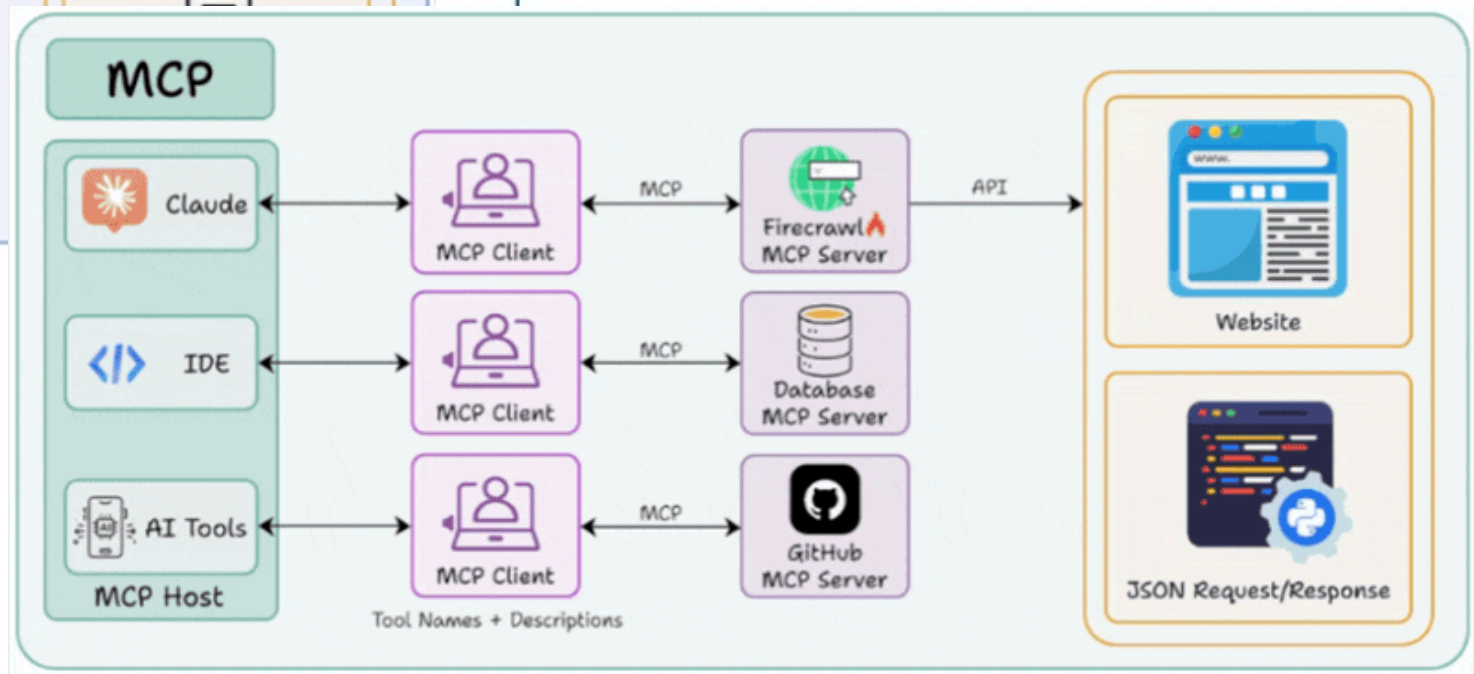
https://github.com/RyanAlberts/best-of-Agent-Harnesses/tree/main

AI-assisted research (life)

AI as a super-power







https://www.anthropic.com/news/claude-for-life-sciences

https://github.com/TensorBlock/awesome-mcp-servers/blob/main/docs/healthcare--life-sciences.md

https://intuitionlabs.ai/articles/mcp-servers-biotech-guide

AI-assisted research (life)

AI as a super-power

https://www.youtube.com/embed/wZvvDEkYfic?enablejsapi=1

https://www.youtube.com/embed/XhKbq8Jzp0U?enablejsapi=1

https://www.anthropic.com/news/claude-for-life-sciences

https://github.com/TensorBlock/awesome-mcp-servers/blob/main/docs/healthcare--life-sciences.md

https://intuitionlabs.ai/articles/mcp-servers-biotech-guide

AI-assisted coding

Live demo preview

🧬 Real data, short paper

Single-cell RNA-seq of 700 human blood cells (PBMCs), 765 genes.

1. How many distinct cell populations are there?
2. Which genes mark each one, and does it match immune biology?
3. How well do unsupervised clusters agree with the reference labels?

🔁 The loop

Plan: the agent proposes, we review

Approve: a human decides

Execute: it writes and runs the code

Verify: numbers checked against script output

Report: a short, reproducible write-up

While you watch, ask: what would you check?

AI-assisted coding

The prompt

Question: using the real single-cell RNA-seq dataset that ships with scanpy (700 human blood cells, 765 genes), how many distinct cell populations are there, which genes mark each one and do they match known immune biology, and how well do unsupervised clusters agree with the reference cell-type labels?

Install what you need with pip. Do not download any other data. **Check what preprocessing the data already had** before applying any.

Write one script, **analysis.py**, that regenerates every figure and number. Deliverable: **report.md**, at most 5 figures, every number traceable to the script's output, limitations section.

First propose a plan: steps, methods and why, main risks. **Do not run anything until I approve it.**

Plan, review, approve, then execute end to end

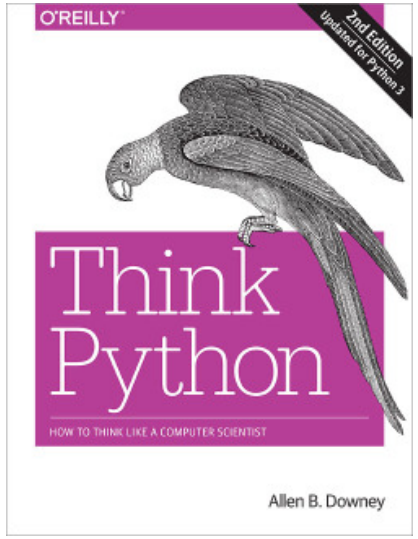
Practicalities

Taking Python for a spin

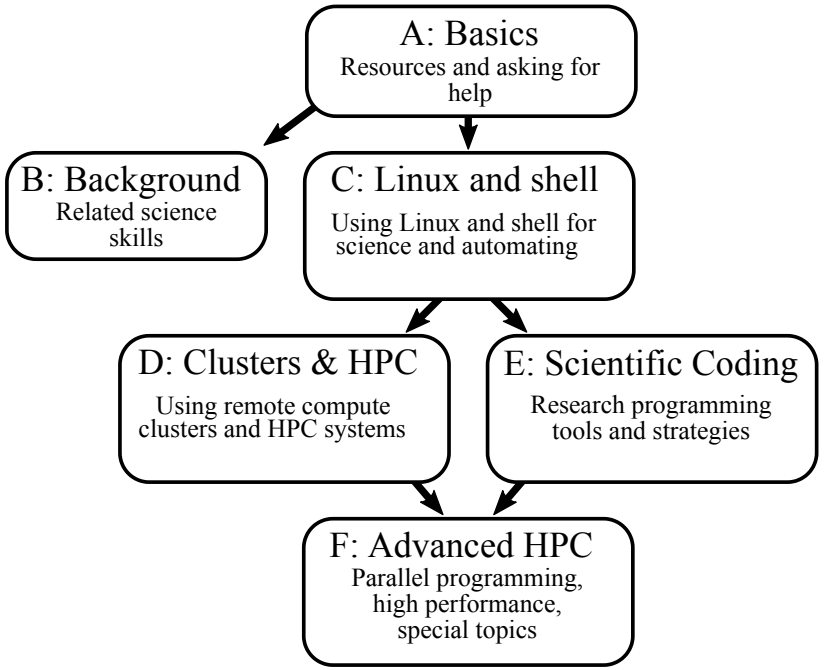
https://www.online-python.com/

- W3School demo compiler
- Trinket
- Online-python

Practicalities



2nd Edition Free online



Aalto University "Scientific Computing" course

Online Resources

datacamp

class central

freeCodeCamp

UDACITY

SKILL SHare.

- The internet is filled with Python tutorials, materials and exercises
- Always check official documentation
- Remember: *practice makes perfect*

Practicalities

- Make sure you have the latest materials
- Ask for help if you get stuck!
- Errors are not evil! Don't panic!
- Errors tell you:
  - *What* went wrong
  - *Where* it went wrong

```
In [1]: run myerror.py
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
/Users/morrison/teaching/IntSciProWS12/myerror.py in <module>()
      1 x = 2
      2 y = "hello"
----> 3 print x + y

TypeError: unsupported operand type(s) for +: 'int' and 'str'

In [2]:
```

Practicalities

Practical exercises

- Use online python consoles to practice
- If running locally, you can use iPython / JuPyTer for interactive programming (more on that later)

```
neurobook@neurobook:~$ python
Python 3.11.5 (main, Sep 11 2023, 13:54:46) [GCC 11.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> x = 5
>>> y = 10
>>> x*y
50
>>> y/3.7
2.7027027027027026
>>>
```

- we can program interactively in ipython
- We can iteratively transfer our code to text files (scripts) when they seem to be doing the right thing so we can re-use the code later
- The script can also be run from within ipython
- Then everything we define in the script is available in the interpreter: [1] run myscript.py
- This helps with debugging and further development of the program

```
neurobook@neurobook:~$ ipython
Python 3.11.5 (main, Sep 11 2023, 13:54:46) [GCC 11.2.0]
Type 'copyright', 'credits' or 'license' for more information
IPython 8.15.0 -- An enhanced Interactive Python. Type '?' for help.

In [1]: x = 5

In [2]: y = 10

In [3]: x*y
Out[3]: 50

In [4]: y/3.7
Out[4]: 2.7027027027027026

In [5]:
```

Thank you!





Course materials (Part I) adapted from:

- Moritz Helias
- Abigail Morrison
- Yury Zaitsev
- Renato Duarte



